



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

**Applied Machine Learning: From First Principles to Production Systems in the
African Fintech Ecosystem**

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with L^AT_EX using the book class.
Figures produced with TikZ; code listings with listings.

Contact: jgarcia@strathmore.edu

Version 0.1 — August 19, 2026

Contents

Preface	ix
I Foundations and Classical Machine Learning	1
1 Introduction to Machine Learning	3
1.1 Why This Chapter Comes First	4
1.2 What Is Machine Learning?	5
1.2.1 An operational definition	5
1.2.2 ML inverts the direction of programming	6
1.2.3 When ML earns its keep — and when it does not . . .	7
1.2.4 The formal core: empirical risk minimization	7
1.2.5 The Bayes-optimal reference	10
1.2.6 Notation for the rest of the book	11
1.3 The Machine-Learning Landscape	11
1.3.1 The four learning paradigms	11
1.3.2 Two orthogonal operational axes	13
1.3.3 No free lunch, and what it licenses	15
1.4 Why ML Systems Fail in Production	16
1.4.1 The small green box	16
1.4.2 The notebook-to-production gap, localized	17
1.5 The End-to-End ML Lifecycle	18
1.5.1 From CRISP-DM to a monitoring-driven loop	18
1.5.2 It is a loop, not a waterfall	19
1.6 Framing the Problem and Choosing the Metric	20
1.6.1 Step 1 is where projects are won	20
1.6.2 Regression metrics: how the business pays for errors	21
1.6.3 Classification under imbalance: why accuracy lies . .	21
1.7 A First Look at the Data	24
1.7.1 The raw material	24
1.7.2 A feature dictionary, declared up front	24
1.7.3 What to interrogate, and why	25
1.8 Data, Splits, and Leakage	27

1.8.1	Splits: the discipline that protects your estimate of $R(f)$	27
1.8.2	Data leakage: the silent killer of fintech models	28
1.9	Baselines, Evaluation, and Generalization	30
1.9.1	Why generalization is hard: the bias–variance decomposition	30
1.9.2	Always ship a baseline first	31
1.9.3	Cross-validation: cheap insurance on the estimate . .	31
1.10	Presenting and Deciding	33
1.10.1	The decision memo	33
1.10.2	Champion, challenger, and shadow mode	33
1.11	Fairness, Explainability, and Regulation	34
1.11.1	Making “fair” precise enough to test	34
1.11.2	Explainability as a deployment requirement	35
1.12	From Model to System: MLOps and the Agentic Outlook . .	35
1.12.1	The MLOps spine of this course	36
1.12.2	Where LLMs and agents fit — and where they do not	37
1.13	Chapter Summary	38
	Exercises	41
	Key Terms	42
	Further Reading	42
2	Data Preparation and Feature Engineering	45
2.1	Introduction: From Raw Data to Model-Ready Data	46
2.1.1	The Six Dimensions of Data Quality	47
2.1.2	The Unit of Prediction	48
2.2	Missing Data	48
2.2.1	Rubin’s Taxonomy of Missingness	49
2.2.2	Diagnosis Before Treatment	50
2.2.3	Imputation I: Simple Statistics with Indicators	50
2.2.4	Imputation II: Model-Based Methods	51
2.2.5	A Decision Framework	52
2.3	Outliers and Duplicates	53
2.3.1	Detecting Outliers	53
2.3.2	Treating Outliers, and When Not To	54
2.3.3	Duplicates and Entity Resolution	54
2.4	Scaling and Transformations	55
2.4.1	Why Scale, and With What	55
2.4.2	Taming Skew: Log and Power Transforms	56
2.5	Encoding Categorical Variables	57
2.5.1	Low Cardinality: One-Hot and Ordinal	57
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	58
2.5.3	High Cardinality II: Target Encoding	58
2.6	Feature Engineering	59

2.6.1	Engineered versus Learned Features	59
2.6.2	Domain Features for Mobile-Money Credit Scoring	60
2.6.3	Datetime Features and Cyclical Encoding	61
2.6.4	Interactions, Binning, and Crosses	61
2.6.5	What Makes a Feature Good?	62
2.7	Data Leakage	63
2.7.1	Definition and Why It Is Dangerous	63
2.7.2	Six Common Causes	64
2.7.3	Temporal and Group-Aware Splits	64
2.7.4	A Detection Protocol	65
2.7.5	The Structural Antidote: Pipelines	66
2.8	From Pipelines to Feature Platforms	67
	Assignment 2 (graded, 20 marks)	68
	Exercises	69
	Key Terms	70
	Further Reading	70
3	Classical Machine Learning and Rigorous Evaluation	71
3.1	Introduction	72
3.2	The Classical Toolbox	73
3.2.1	Why classical models still run production tabular ML	73
3.2.2	Logistic regression	73
3.2.3	Softmax regression for multiclass problems	75
3.2.4	Maximum-margin classification: support vector machines	75
3.2.5	Neighbours and Bayes: two more baselines	76
3.2.6	Choosing among the classical learners	77
3.3	Beyond Accuracy: Metrics that Match the Problem	78
3.3.1	The accuracy trap	78
3.3.2	The confusion matrix	78
3.3.3	Precision, recall, and the F_β family	79
3.3.4	From metrics to money: cost-sensitive evaluation	79
3.4	Ranking Quality: ROC and Precision–Recall Analysis	80
3.4.1	Scores, not labels: the threshold as a free parameter	80
3.4.2	The ROC curve and AUC	81
3.4.3	Precision–recall curves: the imbalance-honest view	82
3.5	Calibration: Do the Probabilities Mean Anything?	82
3.6	Cross-Validation and Honest Uncertainty	84
3.6.1	Why one split is not enough	84
3.6.2	Cross-validating the pipeline	85
3.6.3	Nested cross-validation for model selection	86
3.6.4	Comparing models honestly	86
3.7	Learning from Imbalanced Data	87
3.7.1	Three families of remedies	87

3.7.2	SMOTE: synthesising minority examples	88
3.7.3	The cardinal sin: resampling before splitting	89
3.8	Case Study: Evaluation Design for the Thin-File Credit Scorer	90
3.9	Chapter Summary	91
4	Decision Trees, Ensembles and Hyperparameter Tuning	95
4.1	Introduction	96
4.2	Decision Trees: The Building Block	98
4.2.1	A tree is a sequence of questions	98
4.2.2	The same object, drawn as a partition	98
4.2.3	How a tree is grown	99
4.2.4	Controlling complexity: stopping and pruning	100
4.2.5	The flaw that motivates the rest of the chapter	101
4.3	Why Ensembles Work	102
4.3.1	The wisdom of independent crowds	102
4.3.2	The variance of an average	103
4.4	Voting Classifiers	104
4.5	Bagging and Random Forests	106
4.5.1	Bootstrap aggregating	106
4.5.2	Out-of-bag evaluation	106
4.5.3	Random Forests: decorrelating the trees	107
4.6	Boosting	109
4.6.1	The idea, before the algebra	109
4.6.2	A worked trace	110
4.6.3	From residuals to gradients	111
4.6.4	AdaBoost, the historical special case	112
4.6.5	Shrinkage and early stopping	113
4.7	Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost	113
4.7.1	Regularisation inside the objective	114
4.7.2	The library landscape	115
4.7.3	Monotonic constraints and regulated deployment	115
4.8	Stacked Generalisation	117
4.9	Hyperparameter Optimisation	118
4.9.1	What is a hyperparameter, and why is it different?	118
4.9.2	A ladder of strategies	119
4.9.3	Grid search versus random search	119
4.9.4	Learning from previous trials: Bayesian search and TPE	121
4.9.5	Spending less per trial: successive halving and Hyperband	122
4.9.6	Designing the search space	123
4.9.7	Honest evaluation of a tuned model	123
4.10	Case Study: Round Three of the Credit Scorer	125
	Exercises	127
	Key Terms	128

Further Reading	129
5 Unsupervised Learning and Dimensionality Reduction	131
5.1 Introduction: Learning Without Labels	132
5.2 The Unsupervised Landscape	134
5.3 Clustering with K -Means	135
5.3.1 The objective	136
5.3.2 Choosing k	137
5.3.3 Implementation	139
5.4 Density-Based Clustering: DBSCAN	140
5.5 Gaussian Mixtures and EM	142
5.5.1 The model	142
5.5.2 Fitting by Expectation–Maximisation	143
5.5.3 Selecting k by information criteria	145
5.5.4 Mixtures as anomaly detectors	146
5.6 The Curse of Dimensionality	147
5.7 Principal Component Analysis	148
5.7.1 Variance maximisation	148
5.7.2 Reconstruction-error minimisation	149
5.7.3 Computation and the choice of d	149
5.7.4 Variants you will actually need	151
5.8 Manifold Learning and Visualisation	152
5.8.1 t -SNE	152
5.8.2 UMAP, and the production consideration	153
5.9 Case Study: The Segmentation Pipeline in Production	154
5.9.1 Profiling: turning clusters into personas	154
5.9.2 Stability: a segment that moves is not a segment	154
Exercises	158
Key Terms	160
Further Reading	161
 II Deep Learning and Generative AI	 163
6 Deep Learning Fundamentals	165
6.1 Introduction	165
6.2 Core Theory	165
6.3 Implementation	165
7 Computer Vision with Convolutional Networks	167
7.1 Introduction	167
7.2 Core Theory	167
7.3 Implementation	167

8	Sequence Models and Time Series	169
8.1	Introduction	169
8.2	Core Theory	169
8.3	Implementation	169
9	Transformers, Large Language Models and RAG	171
9.1	Introduction	171
9.2	Core Theory	171
9.3	Implementation	171
III	MLOps and Production Systems	173
10	MLOps: Packaging, Serving, Tracking, CI/CD	175
10.1	Introduction	175
10.2	Core Theory	175
10.3	Implementation	175
11	Model Monitoring, Drift and ML System Design	177
11.1	Introduction	177
11.2	Core Theory	177
11.3	Implementation	177
	Appendices	181
A	Capstone Project	181
B	Solutions to Exercises	183
	Chapter 1	183
	Chapter 2	190
	Chapter 3	195

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in [Appendix A](#).

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	9
9	Transformers, LLMs & RAG	10
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

CHAPTER 4

Decision Trees, Ensembles and Hyperparameter Tuning

Course week 4

Learning Objectives

By the end of this chapter, and of the Week 4 session it accompanies, you should be able to:

- describe a decision tree both as a sequence of questions and as an axis-aligned partition of feature space, score a candidate split by hand, and explain why a single tree is a high-variance model;
- explain, using the bias–variance–covariance decomposition, why and when combining models reduces prediction error, and identify which component each ensemble family attacks;
- construct and contrast parallel ensembles (bagging, Random Forests), sequential ensembles (AdaBoost, gradient boosting) and hybrid schemes (voting, stacking), matching the base learner to the mechanism;
- explain gradient boosting as a sequence of small corrections to what the committee still gets wrong, and read the regularised objective of the modern boosting libraries;
- train LightGBM models with monotonic constraints, class weighting and early stopping that satisfy the governance requirements of a regulated credit product;
- treat hyperparameter tuning as budgeted search rather than folklore, and design a defensible study: search space, sampler (grid, random, Bayesian/TPE), pruning strategy and budget;
- evaluate a tuned model honestly, explaining the optimism of inner-loop scores and the role of nested cross-validation and out-of-time test windows.

4.1 Introduction

Chapters 1–3 assembled, piece by piece, a disciplined workflow: a leakage-safe feature pipeline over messy mobile-money data, a family of classical models, and an evaluation protocol (stratified cross-validation, precision–recall analysis, cost-sensitive thresholds) rigorous enough that we can now trust what our numbers mean. What we have not yet done is push *performance*. The logistic scorecard of Chapter 3 is interpretable, calibrated and regulator-friendly, but it draws a single linear boundary through an 87-dimensional feature space, and it leaves money on the table.

This chapter is about extracting the remaining signal from tabular data, and it rests on three ideas that dominate applied practice. The first is that

the workhorse of tabular machine learning is the humble *decision tree*: individually unimpressive, and unstable enough that no serious lender would deploy one alone, which is precisely what makes the rest of the chapter necessary. The second is that *committees of models outperform their members*, provided the members are made to disagree in useful ways; the entire craft of ensembling is the deliberate manufacture of that disagreement. The third is that once a model family has a dozen interacting knobs (and gradient-boosted trees do), choosing their values by intuition is no longer defensible: *hyperparameter optimisation* (HPO) must itself be treated as a budgeted, validated, reproducible optimisation problem.

The two ideas belong together in one chapter because, empirically, they trade places in the credit-scoring story more often than newcomers expect. As we will see in [Section 4.10](#), careful tuning of a gradient-boosted model buys a larger improvement than the switch of model family that preceded it, an observation with a distinguished pedigree ([Melis et al., 2018](#)). Gradient-boosted decision trees, properly tuned, remain the strongest general-purpose baseline for the medium-sized, heterogeneous, partly-categorical tables that dominate African fintech, health and trade applications, frequently outperforming deep networks on such data ([Grinsztajn et al., 2022](#)). Deep learning earns its place in this course from [Chapter 6](#) onwards, for images, sequences and text; for the loan book, the workhorse we build here will still be the one running in production.

Case Study: Thin-File Credit Scoring, Round Three

We continue the running case of a Nairobi digital lender underwriting 30-day nano-loans from mobile-money behaviour alone (top-up cadence, wallet velocity, agent geography, airtime purchase patterns) for applicants with no credit-bureau file. The dataset, prepared with the pipeline of [Chapter 2](#), contains roughly 1.2 million historical loans described by $d = 87$ engineered features, with a default rate of 4.1% and a strict temporal split: the most recent three months of loans are sealed as an out-of-time test window. The asymmetric economics of [Chapter 3](#) still apply (a missed default costs about KES 10,000 in written-off principal and collection effort, while a wrongly rejected good borrower costs about KES 800 in lost margin and lifetime value), so the business metric remains expected profit per KES 1,000 lent at the cost-optimal threshold, with PR-AUC as the model-development proxy. Round one (Week 3) produced a logistic baseline at ROC-AUC 0.741. In this round we ask how far ensembles and disciplined tuning can push that number *without* sacrificing the governance properties (monotonicity, adverse-action reasons, exclusion of protected attributes) that a regulated lender must preserve.

4.2 Decision Trees: The Building Block

Almost every model in this chapter is a committee of decision trees, so we begin with the member before the committee. A decision tree is also the most direct machine-learning analogue of how a credit officer actually works (a checklist of questions, asked in order), which makes it a good place to build intuition that the rest of the chapter will lean on.

4.2.1 A tree is a sequence of questions

Definition 4.1 (Decision tree). A *decision tree* is a rooted binary tree in which every internal node tests a single feature against a threshold (for numeric features) or a set membership (for categorical ones), and every leaf carries a prediction: a class-probability vector for classification, or a real number for regression. An example is classified by following the path from the root determined by its own feature values, and reading the prediction off the leaf it reaches.

Two properties follow immediately and matter in production. Prediction cost is the depth of the path, not the size of the tree, so even large forests score in microseconds. And the path itself is an explanation: “declined because arrears exceeded seven days *and* wallet balance was below KES 500” is an adverse-action reason in the sense of [Chapter 3](#), generated by the model rather than reconstructed afterwards. [Figure 4.1](#) shows a small tree on the case-study features.

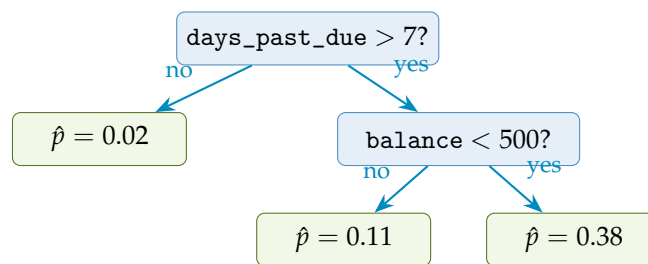


Figure 4.1: A depth-two classification tree on two case-study features. Each leaf reports an estimated probability of default; each root-to-leaf path is a readable decision rule.

4.2.2 The same object, drawn as a partition

The tree diagram is one of two equivalent pictures. Because each test compares one feature against one threshold, every split is a cut perpendicular to a feature axis, and the leaves are disjoint rectangular boxes $R_1, \dots, R_T$

that tile the feature space. The model predicts a constant on each box:

$$F(x) = \sum_{j=1}^T c_j \mathbb{1}\{x \in R_j\}. \quad (4.1)$$

Figure 4.2 draws both pictures of the same model side by side; readers who internalise the correspondence will find the rest of this chapter considerably easier, because ensembles are best understood as operations on partitions.

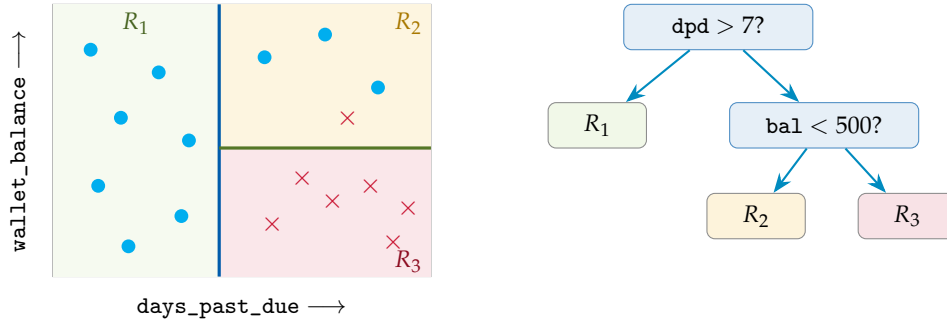


Figure 4.2: One model, two pictures. Left: the leaves as an axis-aligned partition of feature space, with repayers (●) and defaulters (×). Right: the same model as a tree. Boxes and leaves are in one-to-one correspondence.

The geometry explains the tree’s characteristic strengths and weaknesses at a glance. Interactions and thresholds are native: “high arrears *and* low balance” is one box, learned without being named. But a boundary that runs diagonally through the space (the sort a logistic regression captures with one coefficient ratio) must be approximated by a staircase of many splits, which is why trees are weak on smooth linear structure and why the scorecard of Chapter 3 is not simply obsolete.

4.2.3 How a tree is grown

The standard construction is CART (Breiman et al., 1984): grow the tree top-down, one split at a time, always taking the split that looks best right now. At a node holding n rows, the algorithm enumerates every feature j and every candidate threshold s , scores each candidate by the average impurity of the two children it would create,

$$(j^*, s^*) = \arg \min_{j, s} \frac{n_L}{n} I(\text{left}) + \frac{n_R}{n} I(\text{right}), \quad (4.2)$$

applies the winner, and recurses on both children until a stopping rule fires. *Impurity* I measures how mixed a node is: for a node whose proportion of defaults is p , the Gini index is $I_{\text{Gini}} = 2p(1 - p)$ and the entropy is $-p \log_2 p - (1 - p) \log_2 (1 - p)$. Both vanish at $p = 0$ and $p = 1$, where

the node is pure and no further questions are needed, and both peak at $p = \frac{1}{2}$, where the node is useless. Gini is cheaper to compute, is scikit-learn's default, and rarely disagrees with entropy about which split wins. For regression, impurity is the within-node variance and the leaf value is the mean of y .

Example 4.1 (Scoring one split by hand). A node holds 1,000 loans of which 40 defaulted, so $p = 0.04$ and $I_{\text{Gini}} = 2(0.04)(0.96) = 0.0768$. The candidate split `days_past_due > 7` sends 900 loans left (18 defaults, $p = 0.020$, $I = 0.0392$) and 100 right (22 defaults, $p = 0.220$, $I = 0.3432$). The weighted child impurity is

$$0.9(0.0392) + 0.1(0.3432) = 0.0696,$$

so the split improves impurity by $\Delta I = 0.0768 - 0.0696 = 0.0072$. The tree keeps whichever candidate maximises ΔI across all features and thresholds. Note the scale: on a 4%-default problem even a strongly informative split yields a small absolute gain, which is a useful reminder that impurity numbers are internal bookkeeping and not business metrics: the model is still judged by the PR-AUC and expected-profit machinery of [Chapter 3](#).

The word *greedy* in [Eq. \(4.2\)](#) carries weight. The best split now is never revisited later, even if a slightly worse first split would have enabled a much better pair of second splits. Finding the globally optimal tree is computationally intractable, so every practical tree learner surrenders to greediness and, as [Section 4.2.5](#) shows, pays for it in stability.

4.2.4 Controlling complexity: stopping and pruning

Grown without restraint, a tree will split until every leaf is pure, which on a million noisy loans means memorising the training set. There are two families of remedy. *Pre-pruning* stops growth early via `max_depth`, `min_samples_leaf` or `min_samples_split`; it is cheap but myopic, since a weak split may be the gateway to a strong one. *Post-pruning* grows a large tree and then cuts it back, scoring each candidate subtree T by the cost-complexity criterion

$$R_\alpha(T) = R(T) + \alpha |\tilde{T}|, \quad (4.3)$$

where $R(T)$ is the training error and $|\tilde{T}|$ the number of leaves. At $\alpha = 0$ the full tree wins; as α grows, leaves become expensive and weakly-justified branches collapse. Sweeping α traces out a nested sequence of trees, from which cross-validation selects one, an early and unusually clean example of the regularisation-path idea. Scikit-learn exposes this as `ccp_alpha` ([Listing 4.1](#)).

```
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.tree import export_text

tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,           # how many questions deep
    min_samples_leaf=200,  # leaf estimates need support
    ccp_alpha=1e-4,        # cost-complexity pruning
    class_weight="balanced", # ~4% default rate
    random_state=42,
)
tree.fit(X_train, y_train)

names = list(X_train.columns)
print(export_text(tree, feature_names=names)) # readable rule set

```

Listing 4.1: A single decision tree on the credit data, with the complexity controls that matter and a printout that a reviewer can read.

These settings are our first genuine *hyperparameters*: quantities chosen by the modeller rather than fitted from data. We set them by hand here and by search in [Section 4.9](#).

4.2.5 The flaw that motivates the rest of the chapter

A single tree has low bias (grown deep enough it can fit any training set) and high *variance*. The reason is structural rather than accidental. Because growth is greedy and hierarchical, a small change in the data that flips the root split rewrites every decision beneath it, so two trees fitted to two bootstrap samples of the same loan book can share almost no structure while achieving nearly identical training accuracy ([Fig. 4.3](#)). Instability of this kind is uncomfortable in a regulated setting (the adverse-action reason given to a customer should not be an artefact of which month the model was refitted), and it caps predictive performance, because a high-variance estimator generalises poorly however well it fits.

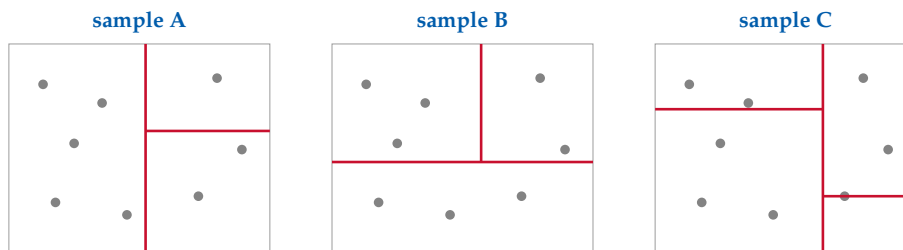


Figure 4.3: Three trees fitted to three bootstrap samples of the same data. The partitions differ radically although the training accuracies barely do. Greedy hierarchical growth converts a small data perturbation into a large model perturbation.

One flaw, two remedies

Every method in the remainder of this chapter is a response to tree variance. If many unstable trees are fitted to perturbed views of the data and *averaged*, the perturbations cancel: that is bagging and Random Forests (Section 4.5). If instead many deliberately *small* trees are added in sequence, each correcting what the partial sum still gets wrong, the committee gains expressive power without any single member being unstable: that is boosting (Section 4.6). Reading trees first makes both look like what they are: disciplined ways of using an unreliable component.

Table 4.1: The single decision tree, honestly scored. The weaknesses column is the agenda for the rest of the chapter.

Strengths	Weaknesses
No feature scaling required; invariant to monotone transforms	High variance; unstable under resampling
Learns interactions and thresholds automatically	Axis-aligned: poor at smooth or linear structure
Handles mixed types; some libraries handle missing values natively	Greedy growth: no global optimality guarantee
Readable: an adverse-action reason falls out of the path	Split search biased towards high-cardinality features
Fast inference, small memory footprint	Rarely competitive on accuracy alone

On the case-study data a depth-four tree of the kind in Listing 4.1 reaches ROC-AUC ≈ 0.712 , below the Chapter 3 logistic scorecard. Taken alone it is not a serious candidate; taken by the thousand, as the next sections show, it becomes the strongest model in the book.

4.3 Why Ensembles Work

4.3.1 The wisdom of independent crowds

The oldest argument for committees predates machine learning by two centuries. Suppose we have M binary classifiers, each of which is correct with probability p on a given example, and suppose (for now, heroically) that their errors are statistically independent. Let the ensemble predict by majority vote.

Proposition 4.1 (Condorcet jury theorem, ML form). *Let $f_1, \dots, f_M$ be classifiers whose errors on input x are independent events, each of probability $1 - p$*

with $p > \frac{1}{2}$, and let M be odd. Then the majority vote is correct with probability

$$P_{\text{maj}} = \sum_{k=\lceil M/2 \rceil}^M \binom{M}{k} p^k (1-p)^{M-k}, \quad (4.4)$$

which is strictly increasing in M and satisfies $P_{\text{maj}} \rightarrow 1$ as $M \rightarrow \infty$.

Proof sketch. The number of correct voters K is a $\text{Binomial}(M, p)$ random variable, and the vote is correct exactly when $K > M/2$, which gives Eq. (4.4). Since $\mathbb{E}[K/M] = p > \frac{1}{2}$, the weak law of large numbers gives $K/M \rightarrow p$ in probability, so the event $\{K/M > \frac{1}{2}\}$ has probability tending to one. Monotonicity in M follows from a standard coupling argument: adding two voters to an odd committee can flip the outcome from correct to incorrect only on a set of smaller probability than the reverse flip. $\square$

Example 4.2. Take $M = 1,000$ voters, each barely better than a coin at $p = 0.51$. Evaluating Eq. (4.4) (or applying the normal approximation with mean 510 and standard deviation $\sqrt{1000 \cdot 0.51 \cdot 0.49} \approx 15.8$) gives $P_{\text{maj}} \approx 0.73$: a committee of near-random experts is a decent classifier. With $p = 0.55$ the same committee exceeds 0.999.

The proposition is both the inspiration and the warning label of ensemble learning. The inspiration is that weak learners suffice. The warning is the independence assumption: models trained by similar algorithms on the same data make *correlated* errors (they are fooled by the same mislabelled loans, the same under-represented customer segments, the same drifting agent behaviour), and correlated committees plateau far below the Condorcet limit. Everything that follows in this chapter is, at bottom, a strategy for lowering that correlation, and the four levers available are: train on different *data* (bagging, Section 4.5), on different *features* (Random Forests, Section 4.5.3), with different *algorithms* (voting and stacking, Sections 4.4 and 4.8), or on different *targets*, namely the errors of the committee so far (boosting, Section 4.6).

4.3.2 The variance of an average

The Condorcet argument concerns classification by vote; a sharper and more general statement is available for averaged predictors, and it makes the role of correlation exact.

Proposition 4.2 (Variance of a correlated average). *Let $f_1, \dots, f_M$ be estimators of the same quantity with common variance $\text{Var}[f_m(x)] = \sigma^2$ and common pairwise correlation $\text{Corr}[f_m(x), f_{m'}(x)] = \rho \geq 0$ for $m \neq m'$. Then the ensemble average $\bar{f}(x) = \frac{1}{M} \sum_{m=1}^M f_m(x)$ satisfies*

$$\text{Var}[\bar{f}(x)] = \rho \sigma^2 + \frac{1-\rho}{M} \sigma^2. \quad (4.5)$$

Proof. Expanding the variance of the sum,

$$\begin{aligned}\text{Var}\left[\frac{1}{M}\sum_m f_m\right] &= \frac{1}{M^2}\left(\sum_m \text{Var}[f_m] + \sum_{m \neq m'} \text{Cov}[f_m, f_{m'}]\right) \\ &= \frac{1}{M^2}(M\sigma^2 + M(M-1)\rho\sigma^2),\end{aligned}$$

using $\text{Cov}[f_m, f_{m'}] = \rho\sigma^2$ for each of the $M(M-1)$ ordered pairs. Collecting terms gives $\sigma^2\left(\frac{1}{M} + \frac{M-1}{M}\rho\right) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$. $\square$

Equation (4.5) deserves to be memorised, because it is the operating manual for half of this chapter. The second term is killed simply by adding members; computation buys it. The first term, $\rho\sigma^2$, is a **floor** that no amount of averaging removes: an ensemble of clones is exactly one model, however large. Consequently there are only two levers on variance, $M \uparrow$ (cheap) and $\rho \downarrow$ (the clever part), and the design of bagging and Random Forests below is best read as a two-step attack on ρ . Note also what the equation does *not* say: averaging leaves bias untouched. Recall from [Chapter 1](#) that expected squared error decomposes into $\text{bias}^2 + \text{variance} + \text{irreducible noise}$; an average of underfit models is still underfit. Reducing bias requires a different mechanism, namely sequential fitting of the committee's own errors, and that is boosting. [Table 4.2](#) summarises the division of labour, and [Fig. 4.4](#) shows [Eq. \(4.5\)](#) at work.

Table 4.2: The ensemble family tree: what each family attacks, and the base learner that suits it.

Family	Trains	Attacks	Base learner
Bagging / Random Forest	parallel	variance	deep trees (low bias)
Boosting	sequential	bias, then variance	stumps, shallow trees
Voting / Stacking	independent	both (heterogeneity)	diverse families

4.4 Voting Classifiers

The simplest way to build a committee is to train several different model families on the same data and combine their outputs directly.

Definition 4.2 (Hard and soft voting). Let $f_1, \dots, f_M$ be classifiers over classes $\mathcal{Y}$. *Hard voting* predicts the modal label,

$$\hat{y}(x) = \arg \max_y \sum_m \mathbb{1}\{f_m(x) = y\};$$

soft voting averages predicted probabilities, optionally with weights, and predicts the class maximising $\hat{p}(y | x) = \frac{1}{M} \sum_m \hat{p}_m(y | x)$.

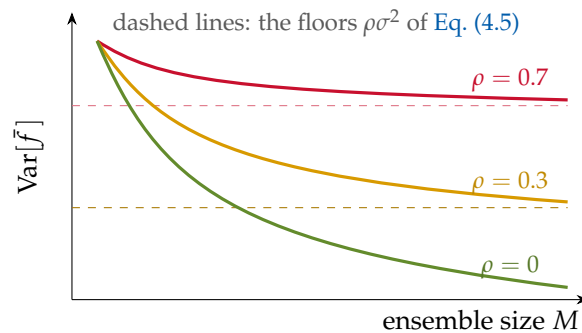


Figure 4.4: Ensemble variance against ensemble size for three values of the pairwise correlation ρ . Adding members exhausts the $\frac{1-\rho}{M}\sigma^2$ term quickly; after that, only decorrelation helps. Random Forests exist to push ρ down.

Soft voting is usually the stronger choice because a probability carries more information than a label: a model that is 95% sure should count for more than one that is 51% sure, and averaging lets it. The caveat, familiar from the calibration analysis of [Chapter 3](#), is that the probabilities being averaged must be *comparable*: mixing a well-calibrated logistic regression with an over-confident boosted model silently hands the boosted model extra votes. Calibrate first, or weight deliberately. In the credit-scoring thread, a logistic scorecard, a Random Forest and a LightGBM model read the top-up and wallet features through genuinely different inductive biases (linear margins, axis-aligned partitions of decorrelated feature subsets, and greedy loss-driven partitions respectively), and their disagreement is exactly the exploitable diversity of [Section 4.3.1](#). [Listing 4.2](#) shows the three-line version; the base estimators are the tuned pipelines of earlier chapters, not raw defaults.

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from lightgbm import LGBMClassifier

vote = VotingClassifier(
    estimators=[
        ("lr", LogisticRegression(max_iter=2000)),
        ("rf", RandomForestClassifier(n_estimators=400)),
        ("lgbm", LGBMClassifier(n_estimators=600)),
    ],
    voting="soft",          # average predicted probabilities
    weights=[1, 1, 2],      # trust the boosted model more
)
vote.fit(X_train, y_train) # X from the Chapter 2 pipeline
```

Listing 4.2: A soft-voting committee over three heterogeneous models. The `weights` argument implements deliberate, rather than accidental, trust allocation.

4.5 Bagging and Random Forests

4.5.1 Bootstrap aggregating

Voting manufactures diversity by changing the algorithm. Bagging (Breiman, 1996) manufactures it by changing the *data*: each member of the committee is trained on a different bootstrap sample of the training set. The instability documented in Section 4.2.5 (the very property that disqualified the single tree) now becomes the raw material, because it means the committee is diverse almost for free.

Definition 4.3 (Bagging). Given a training set $\mathcal{D}$ of n examples, draw M bootstrap samples $\mathcal{D}_1, \dots, \mathcal{D}_M$, each of size n , by sampling from $\mathcal{D}$ uniformly *with* replacement. Train one base learner f_m on each $\mathcal{D}_m$ and aggregate by majority vote (classification) or averaging (regression). The variant that samples without replacement is called *pasting*.

Because the base learners are trained on independent bootstrap draws, the whole procedure is embarrassingly parallel: in scikit-learn a single `n_jobs=-1` spreads the M trees across cores, a luxury that the sequential methods of Section 4.6 will not enjoy. The base learner should be deep, low-bias and high-variance: bagging is a variance reduction device, per Proposition 4.2, and it has no mechanism for repairing bias. Figure 4.5 shows the data flow.

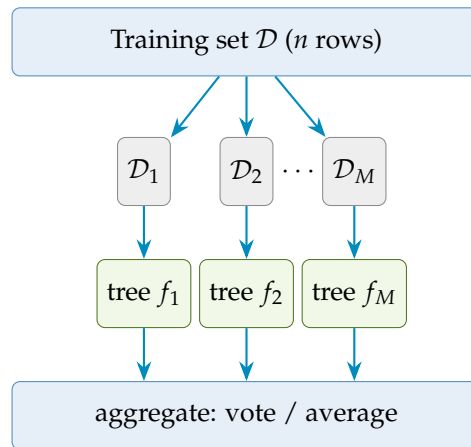


Figure 4.5: Bagging. Each bootstrap sample $\mathcal{D}_m$ is drawn with replacement from $\mathcal{D}$; the M deep trees are trained in parallel and aggregated.

4.5.2 Out-of-bag evaluation

Sampling with replacement has a happy side effect: every tree fails to see a predictable fraction of the data, and those unseen rows constitute a free,

internal validation set.

Proposition 4.3 (The out-of-bag fraction). *The probability that a given row of $\mathcal{D}$ does not appear in a bootstrap sample of size n is $(1 - \frac{1}{n})^n$, which converges to $e^{-1} \approx 0.368$ as $n \rightarrow \infty$. Hence each tree is trained on roughly 63.2% of the distinct rows and leaves the remaining $\approx 36.8\%$ out of bag (OOB).*

Proof. Each of the n independent draws misses the row with probability $1 - \frac{1}{n}$, so all n draws miss it with probability $(1 - \frac{1}{n})^n$. Taking logarithms, $n \ln(1 - \frac{1}{n}) = n(-\frac{1}{n} - \frac{1}{2n^2} - \dots) \rightarrow -1$, so the probability converges to e^{-1} . $\square$

For each row i , aggregate the predictions of only those trees whose bootstrap missed i (about 0.368 M of them) and score the result against y_i :

$$\widehat{\text{err}}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n L(y_i, \bar{f}_{-i}^{\text{oob}}(x_i)). \quad (4.6)$$

Every prediction in [Eq. \(4.6\)](#) is honest (no tree votes on a row it trained on), so the OOB error behaves like a leave-many-out cross-validation estimate obtained at zero additional training cost, which is welcome when a full 5-fold protocol over 800 trees is expensive.

OOB is not out-of-time

The OOB estimate randomises over *rows*, not over *time*. On the loan book, where borrower behaviour, agent networks and macro conditions drift month by month, a random bootstrap freely mixes March into a tree that will be “validated” on February, an exchange the deployed model never gets to make. OOB is a convenient early sanity check; the estimate that governs decisions remains the temporal validation window of [Chapter 3](#), and the sealed out-of-time test set is touched exactly once. This distinction returns with force in [Section 4.9.7](#).

4.5.3 Random Forests: decorrelating the trees

Bagged trees are diverse, but not diverse enough. A dominant predictor (in our data, `days_past_due` on a prior loan) will be chosen at or near the root of nearly every tree, whatever the bootstrap, and trees that agree about their first split agree about a great deal. The pairwise correlation ρ of [Eq. \(4.5\)](#) stays high and the variance floor with it. Breiman’s second idea ([Breiman, 2001a](#)) attacks ρ directly.

Definition 4.4 (Random Forest). A Random Forest is a bagged ensemble of decision trees in which, at *every split of every tree*, the split search is restricted to a fresh uniformly random subset of k of the d features (a common default

is $k = \lceil \sqrt{d} \rceil$ for classification). *Extremely randomised trees* (Extra-Trees) additionally draw the candidate split thresholds at random instead of optimising them.

Feature subsampling forces different trees to build their upper levels from different variables (one tree opens with delinquency features, another with wallet-balance features, a third with agent geography), so their errors decorrelate and, by [Proposition 4.2](#), the ensemble variance falls even at fixed M . The cost is a small increase in the bias of each tree (it is sometimes denied its best split), a trade that deep trees can afford. In practice three knobs matter. The number of trees `n_estimators` is not really a tuning parameter: more trees never hurt accuracy, the curve simply plateaus, so it is set as large as the compute budget allows. `max_features` ($= k$) trades member bias against decorrelation. And `min_samples_leaf` regularises the individual trees; on a 4.1%-default dataset it also guarantees that leaf probability estimates rest on adequate support, which matters because our thresholding logic consumes probabilities, not labels. [Listing 4.3](#) configures the forest accordingly.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=800,
    max_features="sqrt",
    min_samples_leaf=20,      # leaf probabilities need support
    class_weight="balanced",  # ~4% default rate
    oob_score=True,          # free honest estimate (row-wise!)
    n_jobs=-1, random_state=42,
)
rf.fit(X_train, y_train)
print(f"OOB accuracy: {rf.oob_score_:.3f}")
```

Listing 4.3: A Random Forest configured for the imbalanced credit problem: balanced class weights, supported leaves, OOB score as a free sanity check.

Impurity importance loves high-cardinality features

Scikit-learn's `feature_importances_` attribute sums the impurity reduction attributed to each feature over all splits, measured on the *training* data. A high-cardinality identifier such as `agent_id` offers thousands of candidate split points and can always find one that reduces training impurity, so it accumulates a large importance score while carrying little generalisable signal, the same mechanism that made target leakage so seductive in [Chapter 2](#). The audit-grade alternative is *permutation importance* computed on a held-out set: shuffle one feature, remeasure the validation metric, and attribute the drop to the feature. In our data, `agent_id`-style features fall from the top of the impurity ranking to near the bottom of the permutation ranking; the features a credit officer would expect (delinquency, balance dynamics) rise to the top. Report permutation importances, on the temporal validation window, with repeats:

```
permutation_importance(rf, X_val, y_val,
                       scoring="roc_auc", n_repeats=10)
```

4.6 Boosting

Bagging and forests train their members independently and in parallel; each tree is oblivious to the rest of the committee. Boosting inverts the premise: members are trained *sequentially*, and each new member is fitted to whatever the committee has so far got wrong. Where bagging averages away variance, boosting chips away at *bias*, and it does so with base learners that are deliberately weak: stumps, or trees of depth two to four. The sequence, not the individual, carries the expressive power.

4.6.1 The idea, before the algebra

A useful picture is a golfer on a putting green. The first putt does not attempt the whole distance in one heroic swing; it covers most of it and leaves a shorter putt. The second putt aims at *the remaining distance*, not at the hole from the original position. Each stroke is deliberately gentle, because a hard stroke that overshoots leaves a worse position than a soft one that falls short. And the golfer stops when the ball is close enough, rather than continuing to putt for its own sake.

Every element of that picture is a component of gradient boosting. The remaining distance is the *residual*: what the model so far still gets wrong. Aiming at it is *fitting the next tree to the residuals*. The deliberate gentleness is the *learning rate*, and overshooting is overfitting. Stopping when close enough is *early stopping*. [Figure 4.6](#) redraws the same idea in the vocabulary

of models.

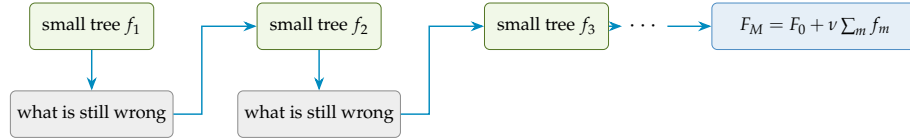


Figure 4.6: Boosting as a sequence of corrections. Bagging asks M trees the same question and averages the answers; boosting asks each tree a new question (“what does the committee so far still get wrong?”) and adds the answers.

Two consequences of the sequential structure are worth stating before any formalism. First, boosting cannot be parallelised across members the way bagging can: tree m cannot be built until tree $m - 1$ exists. (The libraries parallelise *within* a tree instead, across features and histogram bins.) Second, because boosting drives bias down aggressively, it will overfit if allowed to; the regularisation devices in this section (small steps, shallow trees, subsampling, explicit penalties, early stopping) are part of the method’s practical definition rather than optional extras.

4.6.2 A worked trace

Nothing demystifies boosting faster than doing three rounds by hand. [Table 4.3](#) traces a regression example with six loans, a squared-error loss, single-split stumps as base learners, and a learning rate of $\nu = 0.5$.

Table 4.3: Two rounds of gradient boosting on six observations, with $\nu = 0.5$. F_0 is the mean of y ; the stump f_1 splits rows $\{1, 2, 3\}$ from $\{4, 5, 6\}$ and predicts the mean residual on each side; $F_1 = F_0 + \nu f_1$. The residual column then feeds round two.

i	y_i	F_0	$r_i^{(1)} = y_i - F_0$	f_1 leaf	F_1	$r_i^{(2)}$
1	12	13.3	−1.3	−1.30	12.65	−0.65
2	15	13.3	+1.7	−1.30	12.65	+2.35
3	9	13.3	−4.3	−1.30	12.65	−3.65
4	20	13.3	+6.7	+1.37	13.98	+6.02
5	14	13.3	+0.7	+1.37	13.98	+0.02
6	10	13.3	−3.3	+1.37	13.98	−3.98
Sum of squared errors			79.3			71.3

Three observations from the table generalise. The initial model F_0 is merely the best constant: the mean under squared error, the log-odds of the base rate under log-loss. The stump is a very poor model of the residuals, capturing only their split into two groups, and this is fine: boosting asks each member for a crude improvement, not a good model. And because $\nu = 0.5$ the committee moves only halfway towards the stump’s re-

ommendation, which is why the error falls from 79.3 to 71.3 rather than further: deliberate under-correction, exactly as on the putting green. Iterating produces the sharpening staircase of Fig. 4.7.

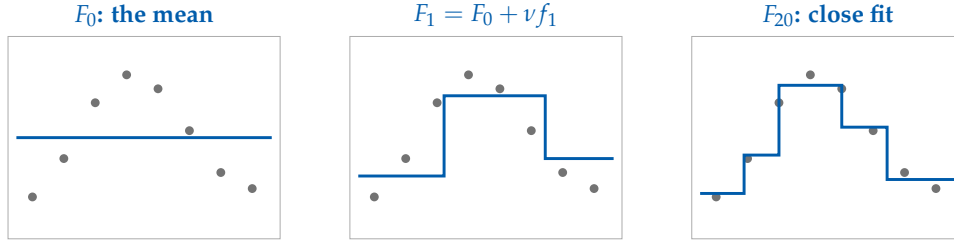


Figure 4.7: Gradient boosting under squared error, in one dimension. The same data appear in each panel; each stage adds a small tree fitted to the current residuals, and the additive staircase converges on the signal. Expressiveness comes from *many shallow* trees, not one deep one; left unchecked, it will eventually fit the noise too.

4.6.3 From residuals to gradients

The trace above fitted plain residuals because the loss was squared error. Gradient boosting (Friedman, 2001) generalises it with a single observation: for the squared-error loss $L = \frac{1}{2}(y - F)^2$, the residual is exactly the negative derivative of the loss with respect to the prediction,

$$-\frac{\partial L}{\partial F(x_i)} = y_i - F(x_i).$$

So if we replace “fit the residual” with “fit the negative gradient of the loss”, the same procedure works for *any* differentiable loss. The quantity being fitted is called the *pseudo-residual*,

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}, \quad (4.7)$$

and the whole algorithm is three lines: start from the best constant F_0 ; at each stage compute the pseudo-residuals Eq. (4.7) and fit a small tree f_m to them; update with a damped step

$$F_m(x) = F_{m-1}(x) + \nu f_m(x), \quad \nu \in (0, 1]. \quad (4.8)$$

The additive form

$$F_M(x) = F_0(x) + \sum_{m=1}^M \alpha_m f_m(x) \quad (4.9)$$

Table 4.4: Common losses and the quantity each stage of gradient boosting actually fits. Choosing the loss is choosing what “still wrong” means.

Loss	Pseudo-residual r_i	Reading
Squared error	$y_i - F(x_i)$	fit the residuals
Binary log-loss (logit F)	$y_i - \sigma(F(x_i))$	fit the probability error
Absolute error	$\text{sign}(y_i - F(x_i))$	fit only the direction

is what results. Table 4.4 lists the pseudo-residuals of the losses used in this course; the log-loss row is the one running in the credit model, where the pseudo-residual is simply the error in predicted probability.

The name is now self-explanatory: ordinary gradient descent updates a parameter vector in the direction of the negative gradient, and gradient boosting updates a *function* in the same way, using a small tree as its step. This is the only abstraction in the section, and everything else in this chapter (AdaBoost before it, XGBoost and LightGBM after it) is a variation on how the step is computed or constrained.

4.6.4 AdaBoost, the historical special case

The first practical boosting algorithm, AdaBoost (Freund and Schapire, 1997), predates the gradient view and expresses the same idea through *row weights* rather than residuals: examples the committee currently misclassifies gain weight, so the next weak learner is forced to attend to them.

Proposition 4.4 (AdaBoost). *With labels $y_i \in \{-1, +1\}$ and initial weights $w_i^{(1)} = 1/n$, stage m fits a weak learner f_m to the weighted data, computes its weighted error $\varepsilon_m = \sum_i w_i^{(m)} \mathbb{1}\{f_m(x_i) \neq y_i\} / \sum_i w_i^{(m)}$, assigns it the coefficient $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$, and re-weights the rows by $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i f_m(x_i))$, the final classifier being $F(x) = \text{sign}(\sum_m \alpha_m f_m(x))$. This procedure is stage-wise minimisation of the empirical exponential loss $\sum_i \exp(-y_i F(x_i))$; in particular α_m is the exact minimiser of that loss for the fitted f_m .*

The coefficient formula is worth reading rather than deriving: a learner with ε_m near zero receives a large α_m and speaks loudly; a coin-flip learner with $\varepsilon_m = \frac{1}{2}$ receives $\alpha_m = 0$ and is silenced; and a learner worse than chance receives a negative coefficient, meaning the committee listens to the opposite of what it says. The re-weighting multiplies each misclassified row’s weight by $e^{\alpha_m} > 1$ and shrinks the rest, which is the row-weight translation of “aim at the remaining distance”.

Remark 4.1 (Why we will move on). The exponential loss is also AdaBoost’s weakness: a mislabelled row on which the committee is confidently wrong

receives *exponentially* growing weight, and later stages contort the boundary around it. Operational credit data contains exactly such rows (disputed defaults, reversed transactions, clerical errors), so the fragility is not hypothetical. Gradient boosting keeps the stage-wise additive structure while letting us choose a robust, task-appropriate loss, and it is the form of boosting used everywhere in this course.

4.6.5 Shrinkage and early stopping

The damping factor ν in Eq. (4.8) (the *learning rate*, historically *shrinkage*) is the single most consequential hyperparameter of the method, and it trades directly against the number of stages M . A large ν reaches a given training loss in fewer trees but explores the function space coarsely and generalises worse; a small ν needs more trees and generalises better (Fig. 4.8). Since the two are coupled, tuning both is wasteful. The protocol used in every listing in this chapter is therefore: fix `n_estimators` generously large, tune ν , and let *early stopping* on the temporal validation window choose the effective number of stages. Training halts once validation performance has not improved for a patience window, which converts a dangerous hyperparameter into a monitored quantity.

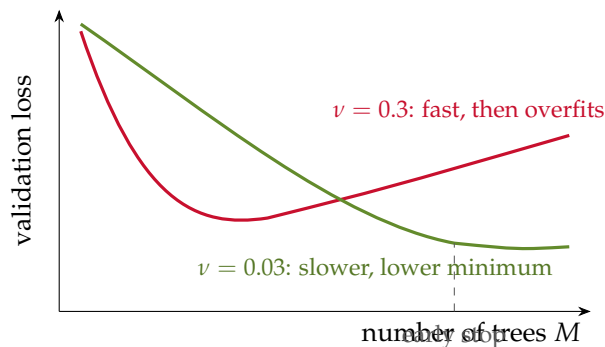


Figure 4.8: Learning rate against tree budget. Small steps take longer and arrive somewhere better; early stopping reads the effective M off the validation curve rather than guessing it.

4.7 Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost

The modern gradient-boosting libraries differ from Section 4.6.3 in two respects that matter to practitioners: they build regularisation into the stage objective itself rather than bolting it on, and they engineer the split search to handle millions of rows. This section derives the first and surveys the second.

4.7.1 Regularisation inside the objective

XGBoost (Chen and Guestrin, 2016) makes two changes to the stage step of Section 4.6.3. First, it uses curvature as well as slope: alongside the gradient $g_i = \partial_F L(y_i, F(x_i))$ it computes the second derivative $h_i = \partial_F^2 L(y_i, F(x_i))$, which turns each stage from a gradient step into a Newton step and lets the algorithm take bigger steps where the loss is flat and smaller ones where it is sharply curved. Second, it charges the new tree for its own complexity, adding to the stage objective the penalty

$$\Omega(f_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2, \quad (4.10)$$

where the tree has T leaves with output values $w_1, \dots, w_T$, $\lambda \geq 0$ is an L2 penalty on leaf values and $\gamma \geq 0$ is a flat price per leaf. The point of Eq. (4.10) is that regularisation is *part of what the tree is optimising*, rather than a post-hoc pruning step as in Eq. (4.3).

Because a tree is constant on each leaf, the resulting objective separates over leaves and each leaf's optimal value can be written down directly. Collecting the gradient and curvature statistics of the rows I_j reaching leaf j as $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$, the optimum is

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad (4.11)$$

and a split of a node into children L and R is worth making only if its *gain*,

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma, \quad (4.12)$$

is positive. Both formulas repay a moment's reading. In Eq. (4.11), every leaf outputs a *score* assembled from gradient statistics rather than a class frequency, which is why boosted trees emit logits that we then calibrate and threshold with the machinery of Chapter 3; and the $+\lambda$ in the denominator shrinks every leaf towards zero, most strongly the leaves holding few rows: an automatic distrust of thinly supported conclusions. Equation (4.12) is structurally the same comparison we made by hand for the Gini index in Example 4.1 (parent score against the sum of child scores), with the $-\gamma$ turning pre-pruning into a by-product of the objective rather than a separate stopping rule. Row subsampling (`subsample`) and per-tree feature subsampling (`colsample_bytree`) then add bagging-style decorrelation on top, so the mature libraries blend both halves of this chapter's story.

4.7.2 The library landscape

The three dominant implementations share Eq. (4.10)–Eq. (4.12) in spirit and differ in growth strategy and categorical handling; Table 4.5 summarises. LightGBM (Ke et al., 2017) grows trees *leaf-wise* (best-first): at each step it splits the leaf with the largest gain anywhere in the tree, rather than completing a level. For a fixed leaf budget this reaches lower training loss than level-wise growth and correspondingly overfits faster, which is why the number of leaves is capped directly (`num_leaves`, kept well below $2^{\text{max_depth}}$) and why `min_child_samples` is a first-class regulariser. Histogram-based split finding and gradient-based sampling make it the fastest of the three on wide tables, and it accepts categorical features natively. CatBoost (Prokhorenkova et al., 2018) contributes *ordered* target statistics for categorical features (each row’s encoding is computed only from rows earlier in a random permutation, a device that removes precisely the target-leakage pathology dissected in Chapter 2), at some cost in raw speed. XGBoost remains the most mature and widely deployed. For the case study we standardise on LightGBM: a million-row, 87-feature table with several engineered categoricals, on laptop-class hardware, is its home ground.

Table 4.5: The three production gradient-boosting libraries.

	XGBoost	LightGBM	CatBoost
Tree growth	level-wise	leaf-wise (best-first)	symmetric (oblivious)
Speed, large tables	fast	fastest	moderate
Categoricals	encode upstream	native	native, ordered statistics
Overfitting profile	moderate	aggressive; <code>num_leaves</code> cap	conservative
Strength	maturity, ecosystem	speed on wide data	leakage-resistant encodings

4.7.3 Monotonic constraints and regulated deployment

A flexible model fitted to finite, noisy data will happily learn locally non-monotone relationships: say, a region in which predicted default risk *decreases* while `days_past_due` increases, because a handful of unusual delinquent-but-recovered borrowers happened to fall there. Statistically this is ordinary variance; institutionally it is a serious defect, because a lender must be able to explain a decline, and “your risk fell when your arrears rose” is not an explanation any review board, customer or regulator will accept.

Definition 4.5 (Monotonic constraint). A model F is *monotonically increasing* in feature x_k if $x_k \leq x'_k$ implies $F(\dots, x_k, \dots) \leq F(\dots, x'_k, \dots)$ with all other features held fixed (decreasing is defined symmetrically). In boosted trees the constraint is enforced during training by rejecting or clamping any split whose child leaf values would violate the required ordering, and all three libraries of Table 4.5 support a per-feature constraint vector with entries $+1$, -1 or 0 .

Constrained models trade a sliver of fit for a shape guarantee, and on well-chosen features the sliver is usually negligible: the true relationship *is* monotone, and the constraint merely refuses the noise. Crucially for this course, the constraint vector is a *governance artefact*, decided by credit policy and documented in the model file; it is not a hyperparameter, and Section 4.9.6 will insist that the HPO sampler never touches it.

Regulatory context: digital credit in Kenya

The Central Bank of Kenya’s Digital Credit Providers framework brings app-based lenders under licensing, conduct and disclosure obligations, and the Kenya Data Protection Act (2019) grants data subjects rights over automated decisions made about them. Together they translate, for a model builder, into concrete artefacts this chapter’s methods must support: risk scores whose direction of response to delinquency and affordability features is documented and monotone (Definition 4.5); adverse-action reasons derivable per declined applicant (permutation importances and, in Chapter 11, SHAP-style attributions (Lundberg and Lee, 2017)); and the exclusion of protected attributes *and their proxies* from the feature set, audited by the correlation screens of Chapter 2. None of this is hostile to performance: it constrains the search to models the institution is actually permitted to deploy.

Listing 4.4 assembles the pieces (constraints, imbalance handling, early stopping) into the configuration that round three of the case study will tune.

```
import lightgbm as lgb

mono = [ +1,    # days_past_due      -> risk must not fall
        -1,    # wallet_balance_kes -> risk must not rise
         0,    # topup_frequency    (unconstrained)
         0 ]   # agent_distance_km   (unconstrained)

clf = lgb.LGBMClassifier(
    n_estimators=4000, learning_rate=0.03,
    num_leaves=63, min_child_samples=100,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=5.0,
    monotone_constraints=mono,           # governance, not a hyperparameter
```

```

    scale_pos_weight=24.0,          # ~ n_neg / n_pos at 4.1% defaults
)
clf.fit(
    X_tr, y_tr,
    eval_set=[(X_val, y_val)], eval_metric="auc",
    callbacks=[lgb.early_stopping(stopping_rounds=200)],
)

```

Listing 4.4: LightGBM for the regulated credit model. The tree budget is deliberately oversized; early stopping on the temporal validation window selects the effective number of stages.

4.8 Stacked Generalisation

Voting (Section 4.4) combines heterogeneous models with fixed, hand-chosen weights. Stacking (Wolpert, 1992) lets the data choose the combination: a *meta-learner* is trained to map the base models' predictions to the target, learning per-model, per-class and, if the original features are passed through, even context-dependent weights. The construction has one non-negotiable subtlety. If the meta-learner were trained on the base models' *in-sample* predictions, it would learn mainly which base model memorises the training set best, a textbook instance of the leakage taxonomy of Chapter 2. The base predictions fed to the meta-learner must therefore be *out-of-fold*: the training set is split into K folds, each base model is trained K times leaving one fold out, and its prediction for every row comes from the copy that never saw that row, the same honesty device as OOB evaluation (Section 4.5.2), realised through cross-validation. Scikit-learn's `StackingClassifier` operates this machinery internally (Listing 4.5); a regularised logistic regression is the customary meta-learner, both for stability on the low-dimensional meta-feature space and because its coefficients read as an interpretable trust allocation over the committee.

```

from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

stack = StackingClassifier(
    estimators=[("lr", lr), ("rf", rf), ("lgbm", gbm)],
    final_estimator=LogisticRegression(C=0.5),
    cv=5,          # out-of-fold machinery
    stack_method="predict_proba",
    n_jobs=-1,
)
stack.fit(X_train, y_train)

```

Listing 4.5: Stacking in scikit-learn. The `cv` argument drives the out-of-fold protocol; `stack_method` feeds probabilities, not labels, to the meta-learner.

The economics of the last basis point

Stacking strictly generalises voting, and on tabular problems it reliably adds something, typically $+0.005$ to $+0.02$ AUC over the best base model, shrinking as the base models converge on the same signal. Whether that something is worth having is not a statistics question. Every base model in the stack must be trained ($K+1$ times), versioned, served, monitored for drift and eventually retired: in production, a stack is several models' worth of operational surface for one model's worth of credit decisions. The mature habit, which [Chapter 10](#) will turn into process, is to price the marginal lift in latency, memory, and maintenance burden before adopting it. Round three's numbers in [Section 4.10](#) give you a concrete instance to argue about, and the lab asks you to take a side in writing.

4.9 Hyperparameter Optimisation

The models of this chapter expose many interacting knobs ([Listing 4.4](#) alone sets seven), and their joint setting moves performance by margins that dwarf most modelling choices. Choosing them by folklore is not defensible; choosing them by exhaustive search is not affordable. This section develops the middle path.

4.9.1 What is a hyperparameter, and why is it different?

Fitting a model means choosing numbers, but not all the numbers are chosen the same way. The split thresholds inside a tree, the leaf values w_j of [Eq. \(4.11\)](#), the coefficients of a logistic regression: these are *parameters*, and they are chosen by the training algorithm from the training data. The tree depth, the learning rate ν , the penalty λ , indeed the choice of model family itself: these are *hyperparameters*, and nothing in the training data chooses them. They are the settings under which fitting happens, so they must be chosen from outside: by us, judged on validation data.

That distinction is the whole difficulty. When we fit parameters we have a gradient telling us which way to move. When we choose hyperparameters we have nothing of the sort. We can only try a configuration, pay for a full training run, and see what score comes back, which makes hyperparameter tuning less like descending a hill and more like mapping a landscape by digging a small number of very expensive holes ([Fig. 4.9](#)). Everything in this section is an answer to one question: *where do I dig next?*

Definition 4.6 (Hyperparameter optimisation). Let $\mathcal{A}_\lambda$ denote a training algorithm under configuration λ drawn from a search space Λ , and let $\mathcal{V}$

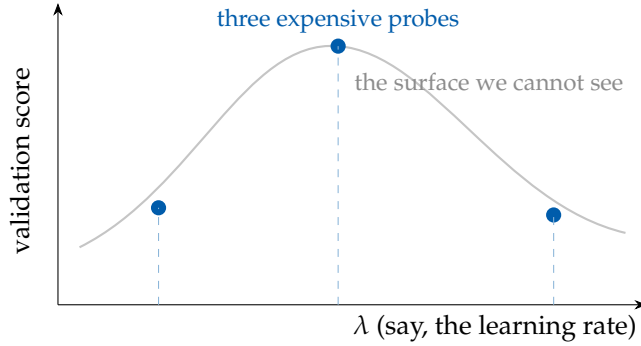


Figure 4.9: Tuning as search in the dark. The objective surface exists but is never observed; each probe costs a full train-and-validate cycle, and the returned score is itself noisy.

be a validation metric. HPO seeks

$$\lambda^* = \arg \min_{\lambda \in \Lambda} \mathcal{V}(\mathcal{A}_\lambda(\mathcal{D}_{\text{tr}}), \mathcal{D}_{\text{val}}), \quad (4.13)$$

subject to a budget B on the number of evaluations (or on wall-clock time, or money).

Three properties of Eq. (4.13) make it awkward (cf. Prince, 2024, §8). Evaluations are *expensive*: one full train-and-validate cycle, minutes to hours. There are *no gradients*, because Λ mixes continuous dimensions (learning rate), integers (num_leaves), categorical (boosting type) and *conditional* dimensions that exist only when others take particular values. And the objective is *noisy*: two runs at identical λ return different scores. Consequently the operative question is never “what is optimal” but “what is the best configuration findable for B trials”, in our setting, for a given spend of compute-hours or KES of cloud credit.

4.9.2 A ladder of strategies

The methods below form a ladder, each rung answering “where do I dig next?” better than the one beneath it, at the cost of more machinery (Fig. 4.10). Climbing it is not automatic: on a small budget the simpler rungs are perfectly respectable.

4.9.3 Grid search versus random search

The classical baseline, grid search, discretises each dimension and evaluates the Cartesian product. Its inefficiency in more than a couple of dimensions has a precise cause (Bergstra and Bengio, 2012): empirically, validation performance has *low effective dimensionality* (it is dominated by a small subset

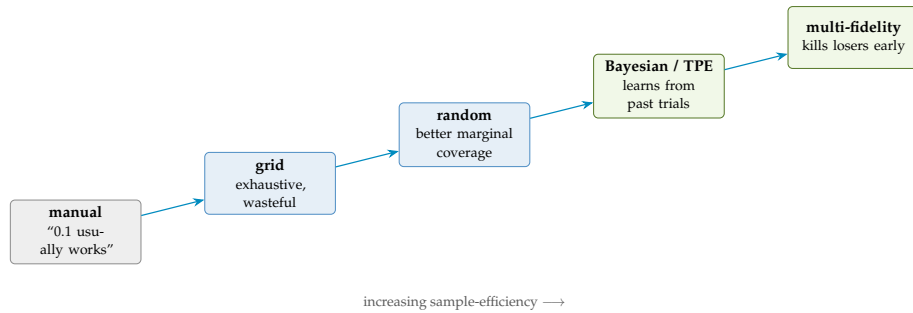


Figure 4.10: The ladder of search strategies. Each rung uses the budget more cleverly than the last; the price is complexity and, from the fourth rung on, a model of the search itself.

of the hyperparameters, different per problem and unknown in advance), and a grid is maximally wasteful in exactly that regime, because its projection onto any single axis collapses onto a handful of repeated values.

Proposition 4.5 (Marginal coverage). *With a budget of B trials in d dimensions, an equal-resolution grid evaluates only $B^{1/d}$ distinct values of each individual hyperparameter, whereas B independent uniform random draws evaluate, with probability one, B distinct values of every continuous hyperparameter.*

The arithmetic is worth feeling rather than proving: if two of six dimensions matter, a $B = 64$ grid probes $64^{1/6} = 2$ values of each important hyperparameter, meaning 62 of the 64 trials re-test important-parameter values already seen, while random search probes 64 values of each (Fig. 4.11). Random search also parallelises trivially and can be stopped at any time, which makes it the floor for any serious study; grid search retains a role only for tiny, discrete, well-understood spaces.

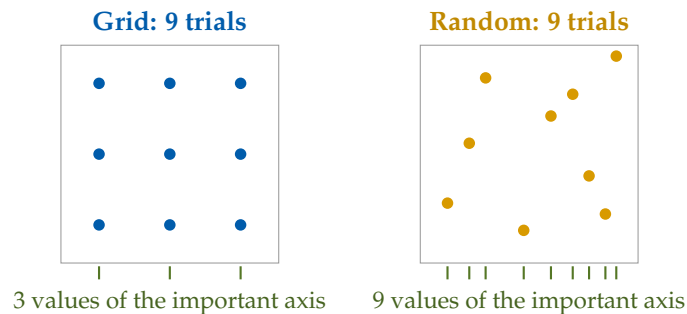


Figure 4.11: Nine trials spent two ways in a two-dimensional space where only the horizontal axis matters. The grid's projections collapse onto three values; the random design's projections cover nine (Proposition 4.5).

4.9.4 Learning from previous trials: Bayesian search and TPE

Random search is memoryless: trial 60 ignores everything trials 1–59 learned about the landscape. When each trial costs real money that waste is worth eliminating, and *Bayesian optimisation* (Snoek et al., 2012) does so with a simple loop. Fit a cheap statistical model (the *surrogate*) to the trials observed so far, so that at every untried configuration we have a guess $\mu(\lambda)$ and an uncertainty $\sigma(\lambda)$ about it. Then choose the next probe by maximising an *acquisition function*, which scores a candidate on how much it might improve on the best result so far. The standard choice, *expected improvement*,

$$\text{EI}(\lambda) = \mathbb{E}[\max(0, \mathcal{V}^* - \mathcal{V}(\lambda))], \quad (4.14)$$

is large where μ looks good, where σ is large, or both. Then evaluate the real objective there, add the result to the history, and repeat. The acquisition function is doing the explore–exploit arbitration that a human tuner does by instinct, and doing it consistently.

The *tree-structured Parzen estimator* (TPE) (Bergstra et al., 2011) reaches the same behaviour by a route that is easier to picture. Rank the trials so far and cut them at a quantile: the top slice is “good”, the rest “bad”. Now build two density estimates over the search space ($\ell(\lambda)$ over the configurations that worked and $g(\lambda)$ over those that did not) and propose the candidate maximising the ratio $\ell(\lambda)/g(\lambda)$ (Fig. 4.12). In words: *propose values that look like the ones that worked and unlike the ones that did not*. That rule can be shown to maximise expected improvement under the model, so TPE is Bayesian optimisation wearing different clothes; because its densities are built dimension by dimension along the tree of conditional parameters, it copes with the mixed and conditional spaces that defeat plain Gaussian-process surrogates, which is why it is the default sampler in Optuna (Akiba et al., 2019). A serviceable rule of thumb: model-based search begins to repay its overhead once the trial count reaches roughly ten times the number of tuned dimensions; below that, random search is honest work.

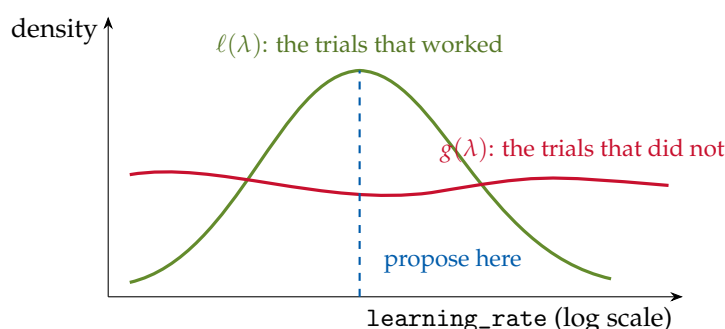


Figure 4.12: The TPE rule. Two densities are fitted to the good and bad trials seen so far, and the next configuration is drawn where their ratio ℓ/g is largest.

4.9.5 Spending less per trial: successive halving and Hyperband

The methods so far treat each trial as all-or-nothing. But a boosted model reveals a great deal about its final quality after a fraction of its boosting rounds, and a configuration clearly losing at round 200 rarely wins at round 2,000. *Successive halving* exploits this: start N configurations, each on a small budget b ; evaluate; keep the best $1/\eta$ fraction and multiply their budget by η ; repeat until one survivor holds the full budget. With $N = 81$, $\eta = 3$ and rungs at $b, 3b, 9b, 27b, 81b$, the same number of configurations at each rung consumes the same total compute, so 81 configurations are screened for about the price of five full-budget runs per rung level. The blind spot is the aggressiveness choice itself: a slow starter is executed at the first rung. Hyperband (Li et al., 2017) hedges by running several brackets of differing aggressiveness in parallel.

In Optuna these ideas surface as *pruners* (the MedianPruner and HyperbandPruner): the objective reports intermediate validation scores as training proceeds, and the framework kills trials that fall below the running median at comparable budget. In routine tabular work, pruning multiplies the effective trial budget by a factor of three to ten, and it composes freely with the TPE sampler, the configuration used in Listing 4.6, which puts the whole of Section 4.9 into twenty-five lines.

```
import optuna, lightgbm as lgb
from sklearn.model_selection import StratifiedKFold, cross_val_score

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3,
        log=True),
        "num_leaves": trial.suggest_int("num_leaves", 15, 255, log=
        True),
        "min_child_samples": trial.suggest_int("min_child_samples", 20, 500,
        log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5,
        1.0),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-2, 50.0,
        log=True),
        "n_estimators": 2000, "scale_pos_weight": 24.0,
        "monotone_constraints": MONO,      # governance is not tunable
    }
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    score = cross_val_score(lgb.LGBMClassifier(**params), X_train, y_train,
        scoring="average_precision", cv=cv, n_jobs=-1)
    return score.mean()      # PR-AUC for the imbalanced target

study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42),
    pruner=optuna.pruners.MedianPruner(n_warmup_steps=5))
```

```
study.optimize(objective, n_trials=100, timeout=3600)
print(study.best_params, study.best_value)
```

Listing 4.6: A complete Optuna study for the credit model: log-aware search space, TPE sampler, median pruning, PR–AUC objective inside stratified cross-validation. Governance parameters are pinned, not searched.

4.9.6 Designing the search space

No sampler can rescue a badly designed space, and space design is craft that precedes compute. Four principles cover most of it. *Log-scale what spans orders of magnitude:* learning rate, regularisation strengths and minimum-leaf sizes live on multiplicative scales, and a uniform prior over $[10^{-3}, 0.3]$ would put almost all its mass above 0.03; the `log=True` flags in Listing 4.6 implement this. *Do not tune redundant knobs:* the tree budget `n_estimators` is fixed high and early stopping selects the effective value, so tuning it alongside the learning rate would waste a dimension re-discovering the coupling of Section 4.6.5. *Freeze what policy owns:* monotonic constraints, protected-feature exclusions and cost weights are governance decisions (Section 4.7.3); a sampler allowed to relax them will, and the resulting “improvement” is a compliance incident. *Narrow as you learn:* after roughly thirty trials, Optuna’s fANOVA-based importance analysis (`optuna.importance`) reveals which dimensions the objective actually responds to, and the space should be tightened around them. Finally, remember that the pipeline of Chapter 2 is part of the model: imputation strategy and encoder choice are categorical hyperparameters like any other, and tuning the whole Pipeline object keeps the comparison leakage-safe. Table 4.6 records the space used for the case study.

Table 4.6: The six-dimensional LightGBM search space of Listing 4.6. By the ten-trials-per-dimension heuristic, 60–100 TPE trials are an appropriate budget.

Hyperparameter	Range	Scale
<code>learning_rate</code>	$10^{-3} - 0.3$	log
<code>num_leaves</code>	15 – 255	log, integer
<code>min_child_samples</code>	20 – 500	log, integer
<code>subsample</code>	0.5 – 1.0	linear
<code>colsample_bytree</code>	0.5 – 1.0	linear
<code>reg_lambda</code>	$10^{-2} - 50$	log

4.9.7 Honest evaluation of a tuned model

Tuning introduces a statistical hazard that the evaluation machinery of Chapter 3 must be extended to meet. The best cross-validated score found

by a search is not an unbiased estimate of the winning configuration's true performance: the search examined many configurations and reported the maximum, and the maximum of many noisy estimates sits above the truth by construction: the *winner's curse* of model selection. Run a hundred trials of identically-good configurations and one of them will still come first, purely by luck of the fold split. The larger the search and the noisier the folds, the worse the optimism.

Reporting the inner-loop best

The number printed by `study.best_value` in Listing 4.6 answers the question “what was the luckiest score my search observed?”, not “how good is my tuned model?” Reporting it as the latter overstates the model, typically by a margin that grows with `n_trials`. Exercise 4.10 quantifies the effect in a setting where it can be computed exactly.

The fully rigorous answer is *nested cross-validation* (Fig. 4.13): an outer loop holds out a test fold, an inner loop (run entirely within the outer training portion) performs the whole search, and the winning configuration is scored once on the outer test fold. The outer scores then estimate the performance of the *procedure* “search, then train the winner”, which is the thing actually being shipped. The cost is a multiplicative K_{outer} on the whole study, which for a hundred-trial search is often unaffordable; the pragmatic industrial compromise, and the protocol of Lab 4, is a single temporal hold-out for tuning plus one sealed out-of-time test window (the last three months of loans) scored exactly once, after all tuning is frozen. Whichever protocol is used, every study is logged (Optuna storage now, MLflow in Chapter 10): an unlogged search is unreproducible archaeology, and reproducibility of the *search* is part of the model's audit trail in a regulated setting.

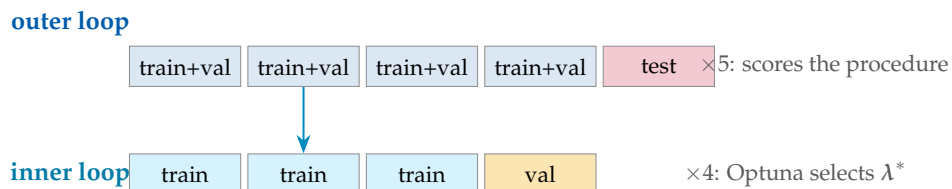


Figure 4.13: Nested cross-validation. The inner loop *selects* the configuration; the outer loop *scores* the whole procedure, search included, on folds that the search never saw.

Three rules of HPO hygiene

First, the test set is touched once, after all tuning is frozen: tuning against the test set is overfitting the test set, however indirect the route. Second, tune the metric the business decision consumes: PR–AUC or expected profit for the 4%-default problem, never accuracy, and not ROC–AUC once the threshold economics of [Chapter 3](#) are in play; the metric you tune is the metric you get. Third, log everything: sampler seed, space definition, every trial, the pruning decisions. These three rules cost nothing and prevent the two most common ways a good model becomes a bad launch.

4.10 Case Study: Round Three of the Credit Scorer

We can now run the round-three experiment promised in the opening case study: the frozen Chapter 2 pipeline, the Chapter 3 evaluation protocol, and this chapter’s model ladder, each rung scored on the temporal validation window with the sealed test window untouched until the end. [Table 4.7](#) shows the ladder as you will reproduce it in Lab 4.

Table 4.7: Round-three results on the temporal validation window. Figures are illustrative of the course dataset; Lab 4 reproduces the table from your own runs.

Model	PR–AUC	ROC–AUC
Single decision tree, depth 4 (Listing 4.1)	0.181	0.712
Logistic scorecard (Chapter 3)	0.212	0.741
Random Forest (Listing 4.3)	0.257	0.779
LightGBM, library defaults	0.271	0.791
LightGBM, Optuna-tuned (Listing 4.6)	0.294	0.812
Stacked (LR + RF + tuned LightGBM)	0.297	0.814

Four readings of the table organise the chapter in retrospect. First, the lone tree is the worst model on the ladder (worse even than the linear scorecard), which vindicates the framing of [Section 4.2.5](#): trees earn their keep only in committee. Second, the ensemble step works as the theory says it should: the forest’s gain over the scorecard is a variance story ([Proposition 4.2](#)), and boosting’s further gain is a bias story ([Section 4.6.3](#)). Third (and this is the number students most often mispredict), *tuning bought more than the model swap*: the Optuna study adds +0.023 PR–AUC over default LightGBM, against +0.014 for the switch from forest to boosting, a pattern with ample precedent ([Melis et al., 2018](#)). The folklore that library defaults are “good enough” is measurably expensive here: at the case study’s economics, the tuned model’s improvement translates into several hundred thousand shillings of expected annual profit per hundred million lent, for

one afternoon of compute. Fourth, the stack's further +0.003 poses exactly the question of the keybox in [Section 4.8](#): it is a real improvement, and it may still be the wrong engineering decision once serving cost and monitoring burden are priced; the lab's decision memo makes you take a side and defend it.

Lab 4: Trees, Ensembles and Optuna (1.5 h)

Notebook `lab04_trees_ensembles_optuna.ipynb`, on the Week-3 frozen pipeline and splits.

1. *One tree first* (15 min). Fit the depth-four tree of [Listing 4.1](#); print `export_text` and read three root-to-leaf paths aloud as adverse-action reasons. Refit on two bootstrap resamples and diff the root split: this is [Fig. 4.3](#) on your own data.
2. *Bagging and forests* (15 min). Train the Random Forest of [Listing 4.3](#); record the OOB score and compare it honestly with the temporal-window score. Compute impurity and permutation importances and explain the rank changes of the `agent_id`-style features.
3. *Boosting* (20 min). Train LightGBM with early stopping on the temporal validation window; add the monotonic constraint vector; quantify the PR-AUC effect of `scale_pos_weight` by sweeping it over $\{1, 12, 24, 48\}$.
4. *HPO* (25 min). Run the 60-trial Optuna TPE study of [Listing 4.6](#) with median pruning; produce `plot_optimization_history` and `plot_param_importances`; tighten the space around the two most important dimensions and run 20 further trials.
5. *Honesty check* (10 min). Unseal the out-of-time test window; score the tuned model once; report the optimism gap between the inner-CV best and the test PR-AUC.
6. *Stretch*. Build the stack of [Listing 4.5](#) and measure its marginal lift.

Deliverable: completed notebook plus a ten-line decision memo recommending, with reasons priced in production terms, which single model the lender should deploy. Due before the Week 5 session.

Exercises

- 4.1 Evaluate Eq. (4.4) for $M \in \{11, 101, 1001\}$ at $p = 0.55$ (a normal approximation is acceptable, with justification). Then repeat the calculation assuming the voters come in ten blocks of perfectly correlated clones, blocks mutually independent. What is the effective committee size, and what does this say about the marginal value of adding trees to a forest whose trees share their root split?
- 4.2 Generalise Proposition 4.2 to a *weighted* average $\bar{f} = \sum_m c_m f_m$ with $\sum_m c_m = 1$, $c_m \geq 0$. Show that under equal variances and equal pairwise correlations the uniform weights $c_m = 1/M$ minimise the ensemble variance, and explain why soft voting with unequal weights (Listing 4.2) can nevertheless be preferable when the equal-variance assumption fails.
- 4.3 Derive the limit in Proposition 4.3 without logarithms, using the inequality $1 - x \leq e^{-x}$, and compute the exact OOB fraction for $n = 20$. A colleague proposes using OOB error to pick the decision threshold for the credit model; give two distinct reasons, one statistical and one temporal, why the threshold chosen this way may misbehave in production.
- 4.4 In the setting of Proposition 4.4, show that after the stage- m reweighting the weak learner f_m has weighted error exactly $\frac{1}{2}$ on the new weights $w^{(m+1)}$; i.e. each learner is “used up” by its own update. What does this imply about the sequence of hypotheses AdaBoost visits?
- 4.5 Derive the pseudo-residual Eq. (4.7) for (a) the binary log-loss with logit link, confirming $r_i = y_i - \sigma(F(x_i))$; and (b) the pinball (quantile) loss at level τ . For (b), explain what a gradient-boosted model of the $\tau = 0.9$ quantile of repayment delay would be useful for in the lender’s collections operation.
- 4.6 Prove that the leaf value Eq. (4.11) under squared error loss with $\lambda = 0$ reduces to the mean residual in the leaf, and that $\lambda > 0$ shrinks it by the factor $H_j / (H_j + \lambda)$ with $H_j = |I_j|$. Interpret the shrinkage in terms of leaves supported by few loans.
- 4.7 The risk model must be monotone increasing in `days_past_due`. A colleague argues the constraint is unnecessary because the unconstrained model’s *average* response to the feature is already increasing. Construct a small counterexample (three or four regions suffice) in which the average response is increasing but an individual appli-

cant is declined with a stated reason that violates monotonicity, and explain the governance failure that results.

- 4.8 You have a budget of $B = 729$ trials over $d = 6$ hyperparameters, of which (unknown to you) only two matter. Using [Proposition 4.5](#), compare the number of distinct values of each important hyperparameter probed by an equal-resolution grid versus random search, and estimate (stating your assumptions) the probability that the grid never evaluates within the best decile of both important dimensions simultaneously.
- 4.9 A successive-halving schedule starts $N = 81$ configurations at $b = 25$ boosting rounds with $\eta = 3$, promoting to a maximum of 2,025 rounds. Tabulate the rungs (configurations and budget at each), compute the total cost in configuration-rounds, and compare it with the cost of running all 81 configurations to full budget. State one realistic scenario in the credit problem where the schedule's assumption (that early performance predicts final performance) fails.
- 4.10 (Winner's curse.) Suppose a search evaluates T configurations whose true validation metrics are all equal to v , and each measurement adds independent noise $\varepsilon_t \sim \mathcal{N}(0, s^2)$. Show that the expected reported best is $v + s \mathbb{E}[\max_t Z_t]$ with Z_t standard normal, and use the approximation $\mathbb{E}[\max_t Z_t] \approx \sqrt{2 \ln T}$ to estimate the optimism for $T = 100$ trials with fold-level noise $s = 0.004$ in PR-AUC. Relate the answer to the optimism gap you measured in Lab 4.

Key Terms

decision tree; axis-aligned partition; leaf; greedy recursive splitting; impurity; Gini index; entropy; information gain; regression tree; pre-pruning; cost-complexity pruning; model instability; ensemble; diversity; majority vote; soft voting; bias-variance-covariance decomposition; bootstrap; bagging; pasting; out-of-bag (OOB) estimate; Random Forest; feature subsampling; Extra-Trees; permutation importance; boosting; AdaBoost; exponential loss; gradient boosting; pseudo-residual; shrinkage (learning rate); early stopping; second-order (Newton) boosting; leaf-wise growth; monotonic constraint; ordered target statistics; stacking; out-of-fold prediction; meta-learner; hyperparameter optimisation; search space; grid search; random search; low effective dimensionality; surrogate model; acquisition function; expected improvement; tree-structured Parzen estimator (TPE); successive halving; Hyperband; pruning; fANOVA importance; nested cross-validation; winner's curse; out-of-time test window.

Further Reading

The primary sources of this chapter are short and readable: Breiman et al. (1984) for CART and cost-complexity pruning, Breiman (1996) and Breiman (2001a) for bagging and forests, Freund and Schapire (1997) for AdaBoost, Friedman (2001) for gradient boosting as functional gradient descent, and Wolpert (1992) for stacking. The library papers (Chen and Guestrin (2016), Ke et al. (2017), Prokhorenkova et al. (2018)) are worth reading for their engineering sections as much as their algorithms. On HPO, Bergstra and Bengio (2012) is the essential argument for random search, Snoek et al. (2012) and Bergstra et al. (2011) develop model-based search, Li et al. (2017) the multi-fidelity view, and Akiba et al. (2019) the framework used in the labs; Prince (2024, Ch. 8) gives a compact modern overview of why the problem is hard. Grinsztajn et al. (2022) examines why tuned tree ensembles still dominate tabular benchmarks, and Melis et al. (2018) documents tuning outweighing architecture, the lesson of Table 4.7. Textbook treatments: Géron (2022, Ch. 6–7) for worked ensemble examples, and Huyen (2022, Ch. 5) for AutoML in production context.

Part II

**Deep Learning and Generative
AI**

Part III

**MLOps and Production
Systems**

Appendices

Capstone Project

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument — framing before formalism, business units before metrics, honesty caveats stated explicitly — that the assignments and the capstone are graded against.

Chapter 1: Introduction to ML and End-to-End ML Projects

Solution to Exercise 1 (Framing). (a) Streaming fraud detection with a 300-alerts/day operations team.

Mitchell specification.

- *T*: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- *E*: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- *P*: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@*k* with $k = 300/\text{day}$, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new

fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of [Section 1.4](#).

(b) Thirty-day readmission from clinical pathway data.

Mitchell specification.

- *T*: at discharge, estimate the probability that the patient is readmitted within 30 days.
- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution to Exercise 2 (ML or not).

(a) Blocking transactions above a regulatory limit: rule. The requirement is a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) SIM-swap account takeover: ML, wrapped in rules. The signal is high-dimensional and behavioural (device changes, dormancy-then-drain

patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion. The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) Routing support tickets by language: ML, but off the shelf. Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median (for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to [Section 1.6.2](#). Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between [Eq. \(1.4\)](#) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is [Eq. \(1.6\)](#). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{\text{FN}}$ (approve), $(1 - \hat{p}) c_{\text{FP}}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{\text{FN}} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{\text{FN}}$;
- decline beats refer iff $(1 - \hat{p}) c_{\text{FP}} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{\text{FP}}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{\text{FN}} c_{\text{FP}}}{c_{\text{FN}} + c_{\text{FP}}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of [Fig. 1.9](#), now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) Number of repayment reminders sent: target leakage. Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production ([Definition 1.3](#)).

(b) Median mobile-money balance over the 90 days before application: safe — *conditional* on discipline. The window $[t_0 - 90\text{d}, t_0)$ respects the as-of contract of [Table 1.6](#). The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) Standard deviation of balance over the loan term: temporal leakage. The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse

(which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) Merchant-ID one-hot with random row-wise splits: group leakage.

Repeat merchants appear on both sides of a random split, so the identifier lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in [Section 1.9.1](#): $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving $\text{Bias}^2 + \text{Variance} + \sigma^2$, which is [Eq. \(1.7\)](#). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```

rng = np.random.default_rng(8401)
n, d = 30_000, 10
t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
X = rng.normal(size=(n, d))
w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
drift = 2.5 * t                             # concept drifts over time
logits = X @ w + drift * X[:, 3] - 3.2
y = rng.binomial(1, 1 / (1 + np.exp(-logits)))

model = Pipeline([("s", StandardScaler()),
                  ("m", LogisticRegression(max_iter=2_000,
                                          class_weight="balanced"))])

# (a) Random shuffled split -- time ignored
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=8401)
pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[: , 1]

# (b) Temporal split -- train on the past, test on the future
cut = int(0.75 * n)                         # rows already time-ordered
pb = (model.fit(X[:cut], y[:cut])
      .predict_proba(X[cut:]))[: , 1]

print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually

uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B's approval rate is strictly lower at every useful threshold. Even in contrived cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in [Section 1.11](#).

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through [Eq. \(1.6\)](#)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as [Section 1.11](#) prescribes.

Solution to Exercise 9 (Hybrid architecture).

A full-marks answer reproduces the topology of [Fig. 1.9](#) with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline ([Listing 1.5](#)), served behind the versioned API of [Listing 1.7](#). It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.
- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent's brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of Eq. (1.6), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer's *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients' rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M | X) = P(M)$ (Definition 2.1). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself (Definition 2.2). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient's own (unobserved) status and risk perception: $P(M | X)$ depends on X_{mis} (Definition 2.3). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per Remark 2.1, the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete

case, and let X_i be its (complete-data) value. MCAR states $P(R \mid X) = P(R)$, i.e. $R \perp X$. The density of X among complete cases is, by Bayes' rule,

$$f(x \mid R = 1) = \frac{P(R = 1 \mid X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random sub-sample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \mid n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} \mid X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X \mid X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X \mid X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to $\text{Var}(\mathbb{E}[X \mid X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X \mid X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits,

balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in [Section 2.3](#)). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of [Section 2.6](#). A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z-score’s μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From [Eq. \(2.7\)](#), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min}) / (x_{\max} - x_{\min})$. Replace the maximum by $10 x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10 x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10 x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median’s scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of [Eq. \(2.8\)](#).

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i \mid \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m \bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha / (\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c \mid \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c \mid \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m \bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)). $\square$

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which is the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model’s. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of [Eq. \(2.10\)](#) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\text{\#collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off:

at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (Chapter 6), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training-serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage

hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: “*Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it.*” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association*, not a *causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ($OR \approx RR$ when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \theta^\top \mathbf{x}$:

$$\ell(\theta) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y) \hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \theta = \mathbf{x}$, $\nabla_{\theta} \ell = (\hat{p} - y) \mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\theta} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1 - \hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in Chapter 6.

Solution to Exercise 3.3. From Eq. (3.11), $F_{\beta} = (1 + \beta^2)PR / (\beta^2 P + R)$.

Limit $\beta \rightarrow 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \rightarrow \infty$: divide numerator and denominator by β^2 :

$$F_\beta = \frac{(1 + \beta^{-2}) PR}{P + \beta^{-2} R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2 \cdot 0.3/1.15 \approx 0.522$ because F_2 rewards this classifier's recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$, i.e.

$$(1 - p) (c_{FP} - c_{TN}) \leq p (c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000 + 9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from $(0, 0)$: each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j . Its height above the FPR axis at that moment is

$$(\text{number of positives already passed})/m_+ = \#\{i : s_i^+ > s_j^-\}/m_+.$$

The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$. Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{1}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{1}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is [Proposition 3.2](#).

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$ of [Eq. \(3.15\)](#).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height π , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03 . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of [Section 3.4.3](#) — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From [Eq. \(3.17\)](#), $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525/0.03 = 17,500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options

are exactly the chapter’s toolkit: aggregate information across stratified folds ([Definition 3.7](#)), attach bootstrap intervals to whatever held-out estimate is reported ([Section 3.6.4](#)), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), [Eq. \(3.19\)](#) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi^2_{1,0.95} = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative ([Section 3.3.1](#)).

Solution to Exercise 3.9. Applying [Eq. \(3.20\)](#) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation ([Example 3.1](#)) built on “the region-A indicator” silently breaks.

SMOTE-NC’s repair. SMOTE-NC interpolates only the *numeric* coordinates via [Eq. \(3.20\)](#), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as [Section 3.7.2](#) already warned for plain

SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate (Section 3.5) before applying Proposition 3.1.

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{w p}{w p + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff w p \geq 1 - p \iff p \geq \frac{1}{1 + w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of Eq. (3.13). This is Elkan (2001)'s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model's outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

Chapter 4: Decision tree, ensembles and hyperparameter tuning

Solution to Exercise 4.1. *Independent voters.* With $p = 0.55$, the number of correct voters is $K \sim \text{Binomial}(M, 0.55)$ and the majority is correct when

$K > M/2$. The normal approximation $K \approx \mathcal{N}(Mp, Mp(1-p))$ is justified for these M by the usual rule $Mp(1-p) \gg 1$ (it equals 2.7, 25.0 and 247.7 respectively; for $M = 11$ the approximation is rough and the exact sum is preferable, but it errs by under two points here). Writing $z = (M/2 - Mp) / \sqrt{Mp(1-p)}$ and $P_{\text{maj}} \approx 1 - \Phi(z) = \Phi(-z)$:

M	$-z$	P_{maj} (normal)	P_{maj} (exact)
11	0.332	0.630	0.633
101	1.005	0.843	0.846
1001	3.164	0.9992	0.9992

The committee converges on certainty, exactly as [Proposition 4.1](#) promises.

Correlated blocks. If the M voters consist of ten blocks of $M/10$ perfectly correlated clones, every clone in a block casts the same vote, so the committee behaves as *ten* independent voters, whatever M is. For $M = 1001$ (≈ 100 clones per block) the relevant computation is the $M = 10$ (even; take $M = 11$ as the odd comparator) row, not the $M = 1001$ row: $P_{\text{maj}} \approx 0.63$, and no increase of M at fixed block structure improves it. The *effective committee size* is the number of independent error patterns, not the number of members. This is [Eq. \(4.5\)](#) in disguise — within-block correlation $\rho = 1$ contributes pure floor — and it is precisely the situation of a forest whose trees share their root split: trees that agree on the first partition of the data are partial clones, adding `n_estimators` buys little, and the productive intervention is decorrelation (`max_features`), not more trees.

Solution to Exercise 4.2. Let $\bar{f} = \sum_m c_m f_m$ with $\sum_m c_m = 1$, $c_m \geq 0$, $\text{Var}[f_m] = \sigma^2$ and pairwise correlation ρ . Then

$$\begin{aligned} \text{Var}[\bar{f}] &= \sum_m c_m^2 \sigma^2 + \sum_{m \neq m'} c_m c_{m'} \rho \sigma^2 \\ &= \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \left(\sum_m c_m \right)^2 \right] = \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \right], \end{aligned}$$

using $\sum_{m \neq m'} c_m c_{m'} = (\sum_m c_m)^2 - \sum_m c_m^2 = 1 - \sum_m c_m^2$. Since $\rho \leq 1$, minimising the variance means minimising $\sum_m c_m^2$ subject to $\sum_m c_m = 1$. By Cauchy–Schwarz (or a Lagrange multiplier), $\sum_m c_m^2 \geq \frac{1}{M} (\sum_m c_m)^2 = \frac{1}{M}$, with equality iff $c_m = 1/M$ for all m : *uniform weights are optimal*, and they recover [Proposition 4.2](#).

The optimality is conditional on the exchangeability assumptions. If the members have *unequal* variances — equivalently, unequal skill, as in [Listing 4.2](#) where a tuned LightGBM is simply a better estimator than the logistic baseline — the same Lagrangian calculation with member variances σ_m^2 (and $\rho = 0$ for clarity) gives optimal weights $c_m \propto 1/\sigma_m^2$: precision weighting. Soft voting with unequal weights is the practical, integer-granularity version of this, and stacking ([Section 4.8](#)) is its data-driven completion — the meta-learner *estimates* the weights instead of assuming exchangeability.

Solution to Exercise 4.3. *The limit without logarithms.* From $1 - x \leq e^{-x}$ with $x = \frac{1}{n}$,

$$\left(1 - \frac{1}{n}\right)^n \leq e^{-1}.$$

For the matching lower bound, apply $1 - x \geq e^{-x/(1-x)}$ (valid for $0 \leq x < 1$, and itself a rearrangement of $1 + u \leq e^u$ with $u = x/(1-x)$) to get $\left(1 - \frac{1}{n}\right)^n \geq e^{-n \cdot \frac{1/n}{1-1/n}} = e^{-n/(n-1)} \rightarrow e^{-1}$. The two bounds pinch the sequence to e^{-1} . For $n = 20$, $\left(\frac{19}{20}\right)^{20} = 0.3585$, already within 2.5% of $e^{-1} \approx 0.3679$ — the asymptotics arrive early, which is why “37% out of bag” is a safe rule of thumb even for modest samples.

Why an OOB-chosen threshold misbehaves. (i) *Statistical:* each OOB prediction $\bar{f}_{-i}^{\text{oob}}(x_i)$ in Eq. (4.6) averages only the $\approx 0.37M$ trees that missed row i , whereas the deployed model averages all M . A mean of $0.37M$ trees is a noisier, differently calibrated score than a mean of M trees, so the probability distribution on which the OOB threshold was optimised is not the distribution the production model emits; with the asymmetric costs of the case study (KES 10,000 vs KES 800), even a small calibration shift moves the cost-optimal cut-off materially. (ii) *Temporal:* the OOB estimate randomises over rows, so its implicit population is the historical mixture of the training window. The threshold’s economics depend on the *future* default base rate and score distribution, which drift; the temporal validation window of Chapter 3 is the only estimate that rehearses the deployment condition. In short, OOB may rank models acceptably, but the decision threshold should be set — and re-set, per Chapter 11 — on out-of-time data.

Solution to Exercise 4.4. After stage m , the new weight mass on the rows that f_m got wrong is

$$W_{\text{wrong}}^{(m+1)} = \sum_{i: f_m(x_i) \neq y_i} w_i^{(m)} e^{+\alpha_m} = e^{\alpha_m} \varepsilon_m W^{(m)}, \quad W_{\text{corr}}^{(m+1)} = e^{-\alpha_m} (1 - \varepsilon_m) W^{(m)},$$

where $W^{(m)} = \sum_i w_i^{(m)}$. Substituting $e^{\alpha_m} = \sqrt{(1 - \varepsilon_m)/\varepsilon_m}$ from Proposition 4.4,

$$W_{\text{wrong}}^{(m+1)} = \sqrt{\frac{1 - \varepsilon_m}{\varepsilon_m}} \varepsilon_m W^{(m)} = \sqrt{\varepsilon_m (1 - \varepsilon_m)} W^{(m)} = W_{\text{corr}}^{(m+1)}.$$

The two masses are equal, so the weighted error of f_m on the updated weights is exactly $\frac{1}{2}$: each learner is rendered useless against the distribution it leaves behind. The implication is that AdaBoost can never profitably select the same hypothesis twice in a row — the re-weighting is an *orthogonalisation* step that forces every stage to contribute something the previous stage did not, which is the sequential counterpart of the decorrelation that bagging obtains by randomisation. (Hypotheses can recur later, once the weight distribution has moved on.)

Solution to Exercise 4.5. (a) *Binary log-loss.* With $y \in \{0, 1\}$, logits F , and $\sigma(F) = (1 + e^{-F})^{-1}$,

$$L(y, F) = -[y \ln \sigma(F) + (1 - y) \ln(1 - \sigma(F))].$$

Using $\sigma'(F) = \sigma(F)(1 - \sigma(F))$,

$$\frac{\partial L}{\partial F} = -\left[y \frac{\sigma'}{\sigma} - (1 - y) \frac{\sigma'}{1 - \sigma}\right] = -[y(1 - \sigma) - (1 - y)\sigma] = \sigma(F) - y,$$

so the pseudo-residual [Eq. \(4.7\)](#) is $r = y - \sigma(F)$: the error in predicted probability, as claimed in [Section 4.6.3](#).

(b) *Pinball loss.* At quantile level $\tau \in (0, 1)$,

$$L_\tau(y, F) = \begin{cases} \tau(y - F), & y \geq F, \\ (1 - \tau)(F - y), & y < F, \end{cases} \quad r = -\frac{\partial L_\tau}{\partial F} = \begin{cases} \tau, & y > F, \\ \tau - 1, & y < F, \end{cases}$$

(the loss is not differentiable at $y = F$; any subgradient value in $[\tau - 1, \tau]$ serves). The boosted model then nudges its prediction up by a constant amount whenever it under-predicts and down whenever it over-predicts, with asymmetric magnitudes τ and $1 - \tau$; the fixed point of that asymmetry is the τ -quantile.

Use in collections. A model of the $\tau = 0.9$ quantile of repayment delay answers, per borrower, “by what day will this loan have been repaid with 90% confidence?” That is the planning quantity a collections operation needs: it sizes the follow-up queue conservatively, sets the day on which an unpaid loan becomes genuinely anomalous (rather than merely late-ish) and therefore worth an agent’s call, and prices expected recovery timelines into cash-flow forecasts. The conditional *mean* delay would understate all three, because delay distributions are heavy-tailed and the cost of under-provisioning is asymmetric — the same cost-sensitivity logic as [Chapter 3](#), applied to a regression target.

Solution to Exercise 4.6. Under squared error $L = \frac{1}{2}(y - F)^2$ we have $g_i = \partial_F L = F(x_i) - y_i = -r_i$ (minus the ordinary residual) and $h_i = \partial_F^2 L = 1$. Hence for leaf j , $G_j = -\sum_{i \in I_j} r_i$ and $H_j = |I_j|$, and [Eq. \(4.11\)](#) gives

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} r_i}{|I_j| + \lambda} \xrightarrow{\lambda=0} \frac{1}{|I_j|} \sum_{i \in I_j} r_i,$$

the mean residual in the leaf — so vanilla gradient boosting ([Section 4.6.3](#)) is recovered as the unregularised special case. For $\lambda > 0$,

$$w_j^* = \frac{H_j}{H_j + \lambda} \cdot \underbrace{\frac{1}{H_j} \sum_{i \in I_j} r_i}_{\text{mean residual}},$$

a shrinkage of the mean residual by the factor $H_j/(H_j + \lambda) \in (0, 1)$. The factor is close to 1 for well-populated leaves and small for thin ones: a leaf holding $H_j = 500$ loans with $\lambda = 5$ keeps 99% of its estimate, while a leaf holding 5 loans keeps only 50%. The ridge penalty is thus an automatic scepticism dial — conclusions drawn from few borrowers are pulled toward “no adjustment” — which is exactly the behaviour one wants in a 4.1%-default portfolio where thin leaves are dominated by noise, and it complements the hard floor that `min_child_samples` imposes in [Listing 4.4](#).

Solution to Exercise 4.7. *Counterexample.* Hold every feature fixed except the delinquency feature `days_past_due` (x , in days), and suppose the unconstrained model’s fitted risk is piecewise constant:

$$F(x) = \begin{cases} 0.10, & 0 \leq x < 10, \\ 0.30, & 10 \leq x < 20, \\ 0.20, & 20 \leq x < 30, \\ 0.50, & x \geq 30. \end{cases}$$

Any reasonable notion of the *average* response is increasing — the least-squares linear fit of F on x has positive slope, and the overall trend runs from 0.10 to 0.50 — yet the model is not monotone: it assigns *lower* risk at $x = 25$ than at $x = 15$.

Governance failure. Consider two applicants identical in every respect except that applicant A has 15 days of arrears and applicant B has 25. Suppose the approval threshold sits at 0.25: A is declined, B is approved. The adverse-action reason issued to A necessarily cites delinquency — it is the only feature in play — yet B, with *strictly worse* delinquency, was approved. The stated reason is therefore falsified by the lender’s own decisions: A’s appeal, a regulator’s file review, or a journalist’s paired test will surface the inconsistency, and under the automated-decision provisions of the Kenya DPA (2019) the lender must stand behind an explanation its model contradicts. The average trend is irrelevant to this failure because adverse-action reasons are given to *individuals*, who live at specific points of the curve, not on its regression line. The monotonic constraint of [Definition 4.5](#) removes the failure mode by construction, at the cost of the (noise-driven) dip — which is precisely the trade described in [Section 4.7.3](#).

Solution to Exercise 4.8. By [Proposition 4.5](#), an equal-resolution grid with $B = 729$ and $d = 6$ uses $r = 729^{1/6} = 3$ distinct values per axis, so each of the two important hyperparameters is probed at just *three* values, each combination repeated across the $3^4 = 81$ settings of the four irrelevant axes. Random search probes 729 distinct values of each important axis.

Probability the grid never enters the best decile of both important axes simultaneously. Model each axis as $[0, 1]$ with the “best decile” an interval of length 0.1 whose location is unknown, and take the grid’s three values per axis

to be evenly spaced with a uniformly random offset, $\{u, u + \frac{1}{3}, u + \frac{2}{3}\}$ with $u \sim \text{U}[0, \frac{1}{3})$ (assumptions stated: independent axes, uniform target location, one-third spacing). Since the target interval $0.1 < \frac{1}{3}$, at most one grid value can fall in it, and the probability that one does is $3 \times 0.1 = 0.3$ per axis. Because the grid is a Cartesian product, hitting the decile marginally on both important axes guarantees a trial at the intersection; the two axes are independent, so

$$P(\text{grid hits both}) = 0.3 \times 0.3 = 0.09, \quad P(\text{grid never does}) = 0.91.$$

For random search, a single trial lands in both deciles with probability $0.1 \times 0.1 = 0.01$, so

$$P(\text{random never does}) = (1 - 0.01)^{729} = 0.99^{729} \approx e^{-7.29} \approx 7 \times 10^{-4}.$$

The grid misses the jointly-best region nine times out of ten; random search finds it essentially always. This is Fig. 4.11 with numbers attached, and it is the practical content of low effective dimensionality (Bergstra and Bengio, 2012).

Solution to Exercise 4.9. With $N = 81$, $b = 25$ and $\eta = 3$, the rungs are:

Rung	1	2	3	4	5
Configurations alive	81	27	9	3	1
Budget each (rounds)	25	75	225	675	2025
Cost of rung	2025	2025	2025	2025	2025

Each rung costs exactly $81 \times 25 = 2025$ configuration-rounds — the $1/\eta$ survival rate and the $\times \eta$ budget growth cancel — so the total is $5 \times 2025 = 10,125$ configuration-rounds. Running all 81 configurations to the full 2,025 rounds would cost $81 \times 2025 = 164,025$: successive halving explores the same 81 candidates for 6.2% of the cost, a $16.2\times$ saving, at the price of trusting rung-1 rankings.

Where the trust fails. The schedule assumes that performance after 25 boosting rounds predicts performance after 2,025. The clearest counterexample in our own chapter is the learning rate (Eq. (4.8)): a configuration with $\nu = 0.02$ is deliberately slow — at 25 rounds it has barely moved from the base rate and looks terrible next to a $\nu = 0.3$ configuration — yet it is exactly the kind of run that attains the *lowest* final validation loss. Aggressive halving systematically executes the low- ν candidates at rung 1 and crowns a fast-starting, earlier-overfitting configuration. Mitigations: hedge across brackets (Hyperband, Li et al., 2017), delay pruning (`n_warmup_steps` in Listing 4.6), or normalise the comparison by learning-curve extrapolation rather than raw score.

Solution to Exercise 4.10. Each measured score is $\hat{\mathcal{V}}_t = v + \varepsilon_t$ with $\varepsilon_t = sZ_t$ and $Z_t \sim \mathcal{N}(0, 1)$ i.i.d. The search reports

$$\max_t \hat{\mathcal{V}}_t = v + s \max_t Z_t \implies \mathbb{E}[\max_t \hat{\mathcal{V}}_t] = v + s \mathbb{E}[\max_{t \leq T} Z_t],$$

since v is common and the maximum commutes with the shift. All T configurations are *equally good* by construction, so the entire excess over v is optimism manufactured by selection. Using the standard extreme-value approximation $\mathbb{E}[\max_{t \leq T} Z_t] \approx \sqrt{2 \ln T}$: for $T = 100$, $\sqrt{2 \ln 100} = \sqrt{9.21} \approx 3.03$, so with fold-level noise $s = 0.004$ in PR-AUC the expected reported best is

$$v + 0.004 \times 3.03 \approx v + 0.012.$$

An HPO study of one hundred trials will thus claim roughly $+0.012$ PR-AUC that is not there, even when tuning does nothing at all. Two readings matter for Lab 4. First, the magnitude is the same order as the optimism gap students typically measure between `study.best_value` and the sealed out-of-time score — part of that gap is drift, but a predictable slice of it is pure winner’s curse, and Table 4.7’s tuned-model row would read ≈ 0.28 rather than 0.294 if the inner-loop best were taken at face value. Second, the formula shows the levers: optimism grows only as $\sqrt{\ln T}$ (slowly — more trials are not catastrophic) but linearly in s , so the cheapest honesty gain is reducing fold noise: more folds, repeated CV, or larger validation windows. And, of course, reporting from data the search never touched (Fig. 4.13) removes the curse entirely — which is the point of the chapter’s final pitfall box.

Chapter 5. Unsupervised Learning and Dimensionality Reduction

Solution to Exercise 5.1. Let $J^*(k)$ denote the optimal inertia with k clusters. Take an optimal solution for k with centroids $\mu_1, \dots, \mu_k$ and assignment c . Build a feasible $(k + 1)$ -cluster solution by choosing any point x_r that is not already alone in its cluster, adding the centroid $\mu_{k+1} = x_r$, and reassigning x_r to it. Every other point keeps its assignment and hence its contribution to Eq. (5.1); the contribution of x_r falls from $\|x_r - \mu_{c(r)}\|^2 \geq 0$ to 0. Therefore $J^*(k + 1) \leq J^*(k)$. At $k = m$ we may place one centroid on each point, giving $J^*(m) = 0$.

Consequently “minimise inertia over k ” always returns $k = m$, which is the degenerate clustering in which every wallet is its own segment: the criterion is vacuous because model complexity is unpenalised. The elbow heuristic is not vacuous because it does not minimise J ; it examines the *rate* at which J falls and looks for the point past which additional clusters

buy little. That is an informal appeal to a complexity penalty, which BIC (Eq. (5.9)) makes formal — and this is precisely why BIC has a genuine interior minimum where inertia does not.

Solution to Exercise 5.2. Partition $C_1 = \{0, 1, 2\}$, $C_2 = \{10, 11, 12\}$. By symmetry it suffices to do C_1 .

- $x = 0$: $a = \frac{1+2}{2} = 1.5$; $b = \frac{10+11+12}{3} = 11$; $s = (11 - 1.5)/11 = 0.8636$.
- $x = 1$: $a = \frac{1+1}{2} = 1$; $b = \frac{9+10+11}{3} = 10$; $s = (10 - 1)/10 = 0.9000$.
- $x = 2$: $a = \frac{2+1}{2} = 1.5$; $b = \frac{8+9+10}{3} = 9$; $s = (9 - 1.5)/9 = 0.8333$.

C_2 mirrors these exactly, so $\bar{s} = \frac{1}{3}(0.8636 + 0.9000 + 0.8333) = 0.8657$.

Partition $C'_1 = \{0, 1\}$, $C'_2 = \{2, 10, 11, 12\}$.

- $x = 0$: $a = 1$; $b = \frac{2+10+11+12}{4} = 8.75$; $s = (8.75 - 1)/8.75 = 0.8857$.
- $x = 1$: $a = 1$; $b = \frac{1+9+10+11}{4} = 7.75$; $s = 0.8710$.
- $x = 2$: $a = \frac{8+9+10}{3} = 9$; $b = \frac{2+1}{2} = 1.5$; $s = (1.5 - 9)/9 = -0.8333$.
- $x = 10$: $a = \frac{8+1+2}{3} = 3.6\bar{6}$; $b = \frac{10+9}{2} = 9.5$; $s = (9.5 - 3.6\bar{6})/9.5 = 0.6140$.
- $x = 11$: $a = \frac{9+1+1}{3} = 3.6\bar{6}$; $b = 10.5$; $s = 0.6508$.
- $x = 12$: $a = \frac{10+2+1}{3} = 4.3\bar{3}$; $b = 11.5$; $s = 0.6232$.

$\bar{s} = \frac{1}{6}(0.8857 + 0.8710 - 0.8333 + 0.6140 + 0.6508 + 0.6232) = 0.4686$.

Comment. The natural partition scores far higher (0.866 versus 0.469), so the silhouette correctly prefers it. More instructive is the single strongly negative value, $s(2) = -0.833$: the point $x = 2$ is on average much closer to C'_1 than to the cluster it was assigned to. This is exactly what the silhouette *diagram* is for. An analyst looking only at the mean would see a mediocre 0.469; an analyst looking at the per-point profile sees immediately *which* point is misassigned and can fix it. Note also that $\bar{s}$ for the bad partition is still positive, because four of six points are content — averages hide misassignment, profiles do not.

Solution to Exercise 5.3. Take the six points, with axis 1 in Kenyan shillings and axis 2 as a count of transactions:

	P_1	P_2	P_3	P_4	P_5	P_6
x_1 (KES)	1000	1100	1200	9000	9100	9200
x_2 (count)	2	40	78	2	40	78

Raw data. Squared distances are dominated by x_1 , whose spread is ~ 8000 against ~ 76 for x_2 . K -means with $k = 2$ therefore splits on value: $\{P_1, P_2, P_3\}$ versus $\{P_4, P_5, P_6\}$. (Check: the within-cluster sum of squares for this split is $2 \cdot (20000 + 2 \cdot 1352.7) \approx 4.5 \times 10^4$, whereas splitting on count gives clusters spanning 8000 in x_1 and costs $\sim 1.3 \times 10^8$.)

Standardised data. After centring and dividing by the sample standard deviations ($\sigma_1 \approx 4271$, $\sigma_2 \approx 34.9$), both axes have unit spread and the configuration becomes a symmetric 2×3 grid at $x_1 \approx \pm 0.94$ and $x_2 \approx \{-1.09, 0, 1.09\}$. The two candidate splits now cost the same by symmetry along x_1 , but the three-level x_2 axis has the larger total spread (± 1.09 against ± 0.94), so K -means prefers to cut along x_2 , returning e.g. $\{P_1, P_4\}$ (low count) against $\{P_2, P_3, P_5, P_6\}$. Perturb the numbers slightly to break the tie in whichever direction you like; the point is that the two answers are different and both are “correct” for their respective metrics.

Which to defend. The standardised one — but the honest defence is not that standardisation is a rule. It is that the raw-data partition answers a question nobody asked: “are these transactions large or small in KES?” is answerable with a single threshold and needs no clustering at all. If the business genuinely wants value tiers, use a threshold and say so. Clustering earns its keep only when several dimensions jointly matter, and that requires them to be commensurable. Where a monetary feature is heavy-tailed, prefer a quantile or log transform to plain standardisation, since a handful of whale wallets will otherwise dominate the standardised axis too.

Solution to Exercise 5.4. Construction. Work in $\mathbb{R}^2$ with $\varepsilon = 1.05$ and $\text{minPts} = 8$. Build two clusters, each a dense blob plus one outlying “antenna” point:

- Cluster A : ten points inside a ball of radius 0.05 centred at $(-0.6, 0)$, plus a single point $p_A = (0, 0)$.
- Cluster B : ten points inside a ball of radius 0.05 centred at $(2.6, 0)$, plus a single point $p_B = (2, 0)$.
- The candidate border point $q = (1, 0)$.

Every blob point is core: it sees the other nine blob points (mutual distance ≤ 0.1) and its antenna (distance $0.6 \pm 0.05 \leq \varepsilon$), so $|N_\varepsilon| = 11 \geq 8$. *Each antenna is core:* p_A sees its blob (ten points at ≈ 0.6), itself, and q (distance exactly $1 \leq \varepsilon$), so $|N_\varepsilon(p_A)| = 12 \geq 8$. *q is not core:* its ε -ball contains only q , p_A and p_B , since both blobs sit at distance $\approx 1.6 > \varepsilon$; thus $|N_\varepsilon(q)| = 3 < 8$. Finally A and B are distinct components, because $d(p_A, p_B) = 2 > \varepsilon$ and q — the only object between them — is not core, so it cannot propagate reachability.

Therefore q is a border point directly density-reachable from p_A and from p_B , which belong to different clusters. DBSCAN assigns q to whichever cluster is expanded first: process p_A before p_B and q joins A ; reverse the visitation order and q joins B . Its label is genuinely order-dependent.

Why this does not undermine reproducibility. By Proposition 5.3, the core-point set and the partition of core points into clusters are order-invariant. The ambiguity therefore never changes the number of clusters, their shape, or the membership of any core point — only the label of a point sitting in a low-density bridge, whose ε -ball is by construction almost empty. Operationally: the segments you name and price are determined by their cores, and a point whose label flips is exactly one about which you should not be making a confident decision. If bit-for-bit determinism is required for audit, fix the input ordering and record it; or use HDBSCAN, which assigns such points by membership strength rather than by visitation order.

Solution to Exercise 5.5. With $k = 2$, $\pi_1 = \pi_2 = \frac{1}{2}$, and shared variance σ^2 , the Gaussian normalising constants cancel in Eq. (5.5):

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x_i - \mu_2)^2}{2\sigma^2}\right)} = \frac{1}{1 + \exp\left(\frac{(x_i - \mu_1)^2 - (x_i - \mu_2)^2}{2\sigma^2}\right)}.$$

Expanding the difference of squares, $(x - \mu_1)^2 - (x - \mu_2)^2 = -2x(\mu_1 - \mu_2) + \mu_1^2 - \mu_2^2 = -2(\mu_1 - \mu_2)\left(x - \frac{\mu_1 + \mu_2}{2}\right)$. Hence

$$\gamma_{i1} = \sigma_{\text{logistic}}(w(x_i - x_0)), \quad w = \frac{\mu_1 - \mu_2}{\sigma^2}, \quad x_0 = \frac{\mu_1 + \mu_2}{2},$$

where $\sigma_{\text{logistic}}(z) = (1 + e^{-z})^{-1}$. So the responsibility is exactly a logistic function of x with **slope parameter** $w = (\mu_1 - \mu_2)/\sigma^2$ and **midpoint** at the arithmetic mean of the two component means, where $\gamma_{i1} = \frac{1}{2}$.

As $\sigma^2 \rightarrow 0^+$ with $\mu_1 \neq \mu_2$ fixed, $|w| \rightarrow \infty$: the logistic curve steepens into a step at x_0 , so $\gamma_{i1} \rightarrow \mathbb{1}\{x_i \text{ nearer } \mu_1\}$ for every $x_i \neq x_0$. This is precisely Proposition 5.5 in one dimension: the midpoint x_0 is the perpendicular bisector (here, a single point), and the soft assignment collapses to K -means' hard nearest-centroid rule. The general lesson is that σ^2 acts as a temperature: large σ^2 gives soft, hedged memberships; small σ^2 gives confident hard ones. Reporting a segment membership of 0.51 versus 0.49 as though it were a fact is a failure to read this parameter.

Solution to Exercise 5.6. Divergence. Take $m \geq 2$ distinct points and the family

$$\theta_\epsilon : \quad \pi_1 = \pi_2 = \frac{1}{2}, \quad \mu_1 = x_1, \quad \sigma_1^2 = \epsilon, \quad \mu_2 = \bar{x}, \quad \sigma_2^2 = s^2 > 0 \text{ fixed.}$$

The first component's density at x_1 is $\mathcal{N}(x_1 \mid x_1, \epsilon) = (2\pi\epsilon)^{-1/2}$, so the contribution of observation 1 to the log-likelihood is at least $\log(\frac{1}{2}(2\pi\epsilon)^{-1/2}) \rightarrow +\infty$ as $\epsilon \rightarrow 0^+$. Every other observation $i \geq 2$ retains the second component's contribution, bounded below by $\log(\frac{1}{2}\mathcal{N}(x_i \mid \bar{x}, s^2)) > -\infty$, which does not depend on ϵ . Hence $\ell(\theta_\epsilon) \rightarrow +\infty$: the likelihood is unbounded above and no global maximiser exists. Geometrically, the fitted model has spiked onto a single data point and explains nothing.

Effect of reg_covar. Adding $\lambda > 0$ to the diagonal replaces σ_j^2 by $\sigma_j^2 + \lambda$, so the achievable density at a point is bounded by $(2\pi\lambda)^{-1/2}$ and the per-observation log-likelihood by $\log(\pi_{\max}(2\pi\lambda)^{-1/2}) < \infty$. Summing over m observations bounds ℓ above, restoring existence of a maximum on the (now compact-in-effect) parameter region, and the degenerate spikes are removed from the search space entirely.

Price paid. Three costs. (i) The estimator is biased: every fitted covariance is inflated by λ , so genuinely tight clusters are reported as looser than they are, and density-based anomaly scores are correspondingly flattened. (ii) The reported likelihood, and therefore BIC, is no longer that of the model you claim to be fitting, so BIC comparisons are only valid at a *fixed* λ . (iii) λ is a scale-dependent hyperparameter: the same value means something different on standardised and unstandardised features, which is one more reason to fix the scaling before the mixture. In practice keep λ at the smallest value that prevents collapse (the scikit-learn default 10^{-6} on standardised data is usually right) and treat a fit that needs a large λ as evidence that k is too big.

Solution to Exercise 5.7. Free parameters split into weights, means and covariances. Weights contribute $k - 1$ (they sum to one); means contribute kn . Covariances:

covariance_type	Covariance parameters	Total p
full	$k \frac{n(n+1)}{2}$	$(k - 1) + kn + k \frac{n(n+1)}{2}$
tied	$\frac{n(n+1)}{2}$	$(k - 1) + kn + \frac{n(n+1)}{2}$
diag	kn	$(k - 1) + 2kn$
spherical	k	$(k - 1) + kn + k$

For $n = 32, k = 5$: $k - 1 = 4$ and $kn = 160$.

- full: $\frac{32 \cdot 33}{2} = 528$ per component, so $p = 4 + 160 + 5(528) = \mathbf{2804}$.
- tied: $p = 4 + 160 + 528 = 692$.
- diag: $p = 4 + 2(160) = \mathbf{324}$.
- spherical: $p = 4 + 160 + 5 = 169$.

With $m = 150,000$, $\ln m = \ln(1.5 \times 10^5) \approx 11.9184$. For full to beat diag on BIC we need

$$p_{\text{full}} \ln m - 2 \ln \hat{L}_{\text{full}} < p_{\text{diag}} \ln m - 2 \ln \hat{L}_{\text{diag}},$$

i.e.

$$\begin{aligned} \ln \hat{L}_{\text{full}} - \ln \hat{L}_{\text{diag}} &> \frac{1}{2}(2804 - 324)(11.9184) \\ &= \frac{1}{2}(2480)(11.9184) \approx \mathbf{14,778}. \end{aligned}$$

Averaged over the sample this is only $14,778/150,000 \approx 0.0985$ nats per observation — about a 10% relative improvement in average likelihood. So with 150,000 wallets, full covariances are a *cheap* upgrade: modest correlation structure within segments will pay for 2480 extra parameters comfortably. Repeat the arithmetic at $m = 2000$ and the required gap becomes ≈ 0.5 nats per observation, at which point diag usually wins. The general moral is that the “right” covariance type is a function of sample size, not of taste.

Solution to Exercise 5.8. Independent coordinates. A short simulation (numpy default generator, seed 0; $m = 1000$ points and one query point drawn from the same distribution) gives ratios of the order

n	2	10	100	1000
i.i.d. coordinates	121	3.04	0.44	0.12
5-factor latent model	179	10.4	8.88	7.55

(means over 20 replications; the $n = 2$ column is enormous and wildly variable because $D_{\min}$ can be arbitrarily small in low dimension, so read the *trend* across columns, not the individual digits). In the i.i.d. row the contrast between nearest and furthest neighbour collapses roughly like $n^{-1/2}$, exactly as Proposition 5.6 predicts from $\text{Var}[D_n^2]/(\mathbb{E}[D_n^2])^2 = \mathcal{O}(1/n)$.

Latent factor model. Generate $u_i \in \mathbb{R}^5$ i.i.d. standard normal and set $x_i = Au_i + 0.05 \eta_i$ with $A \in \mathbb{R}^{n \times 5}$ fixed and η_i small isotropic noise (second row of the table). The ratio now stays around 8–10 from $n = 10$ all the way to $n = 1000$: raising the ambient dimension by two orders of magnitude does essentially nothing, because the distances are governed by the five latent coordinates and the extra measured dimensions carry almost no independent variability.

Explanation. The proof of Proposition 5.6 needs the coordinate contributions δ_t to be (near-)independent, so that $\text{Var}[D_n^2]$ grows like n while $\mathbb{E}[D_n^2]$ grows like n , forcing the relative variance to 0. Under the factor model the δ_t are strongly dependent: the effective, or intrinsic, dimension is 5, not n , and it is the intrinsic dimension that governs concentration. This is the entire justification for the chapter’s strategy. We do not escape the curse by wishing it away; we escape it because real behavioural features are redundant, and PCA finds the small number of directions along

which the data actually varies. It is also why one should measure the explained-variance curve before concluding that a 200-feature problem is high-dimensional: it may have an intrinsic dimension of 20.

Solution to Exercise 5.9. Proof. From the proof of Proposition 5.7,

$$E(W) = \frac{1}{m} \sum_i \|x_i\|^2 - \text{tr}(W^\top S W).$$

For centred data $\frac{1}{m} \sum_i \|x_i\|^2 = \text{tr}(S) = \sum_{t=1}^n \lambda_t$. At the optimum $W = W_d$ we have, by Theorem 5.1, $\text{tr}(W_d^\top S W_d) = \sum_{t \leq d} \lambda_t$. Hence

$$E(W_d) = \sum_{t=1}^n \lambda_t - \sum_{t \leq d} \lambda_t = \sum_{t > d} \lambda_t,$$

as claimed: the mean squared reconstruction error is exactly the variance you threw away.

Numerical verification. With $S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ from Example 5.2, $\lambda_1 = 6$, $\lambda_2 = 2$, $v_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$. For $d = 1$ the formula predicts $E = \lambda_2 = 2$. Take three centred points consistent with S , for instance

$$x_1 = (3, 1)^\top, \quad x_2 = (-1, -3)^\top, \quad x_3 = (-2, 2)^\top.$$

Projection: $z_i = v_1^\top x_i$ gives $z_1 = 4/\sqrt{2} = 2\sqrt{2}$, $z_2 = -4/\sqrt{2} = -2\sqrt{2}$, $z_3 = 0$. Reconstruction $\hat{x}_i = z_i v_1$: $\hat{x}_1 = (2, 2)^\top$, $\hat{x}_2 = (-2, -2)^\top$, $\hat{x}_3 = (0, 0)^\top$. Residuals: $x_1 - \hat{x}_1 = (1, -1)^\top$ with squared norm 2; $x_2 - \hat{x}_2 = (1, -1)^\top$, squared norm 2; $x_3 - \hat{x}_3 = (-2, 2)^\top$, squared norm 8. Mean = $(2 + 2 + 8)/3 = 4$.

The mean over this particular triple is 4, not 2, because three points do not reproduce S : their own sample covariance is $\begin{pmatrix} 14/3 & 2/3 \\ 2/3 & 14/3 \end{pmatrix}$, whose second eigenvalue is $14/3 - 2/3 = 4$ — and 4 is exactly what the formula predicts for *their* covariance. The identity holds sample-wise, and checking it this way is a useful reminder that λ_t are properties of the sample you actually projected, not of an assumed population.

Solution to Exercise 5.10. A defensible one-page answer covers four things.

What to measure next. (i) Whether the disparity is in the *clustering* or in the *limit rule*: compute segment membership rates by county, then, within segment, compare limits — if limits are equal within segment, the mechanism is composition, not pricing. (ii) The behavioural distance between this segment and its neighbours: if the silhouette for this cluster is weak, the segment may be an artefact and the disparity a modelling choice rather than a finding. (iii) Whether the underlying features are themselves proxies: regress county on the top principal components and see how predictable it

is; high predictability means the “behavioural” representation substantially encodes geography. (iv) Outcome data where available — if this segment’s historical default rate genuinely differs, the disparity may be justified; if it does not, the 40% limit reduction lacks a risk basis.

Instruments engaged. Both discussed in the chapter’s regulatory box. The Data Protection Act, 2019 governs the derived segment as personal data: lawful basis, purpose limitation, and the data subject’s rights of access and objection apply to an inferred behavioural profile as much as to the raw transaction log. The CBK Digital Credit Providers Regulations, 2022 require pricing and terms to be disclosed and non-exploitative and credit decisions to be explicable — an unexplainable “cluster 3” limit rule does not meet that standard, and a limit reduction concentrated on two counties invites scrutiny of whether the differentiation is risk-based or merely correlational.

Two remedies with costs. (i) *Re-fit excluding geography-loaded features* (agent-location density, merchant-mix features tied to regional paybills). Cost: some genuine predictive signal is lost, so expected credit losses rise; and the remedy is incomplete, since geography can be reconstructed from residual features. (ii) *Keep the segmentation but sever it from pricing*, using segments only for product design and communication while limits are set by the audited, monotone scorecard of [Chapter 4](#) with fairness constraints. Cost: loses the operational simplicity of segment-based pricing, and requires a second governance process for the scorecard. A third, cheaper option worth listing: cap the maximum limit differential across segments, which bounds the harm without discarding the model.

When shipping unchanged is defensible. If the disparity is *explained* by a legitimate, disclosed, risk-relevant behavioural difference that survives conditioning on geography — for example, the segment is defined by seasonal single-harvest income, the lower limit reflects a measured higher default rate for bullet-repayment products, the county concentration is a consequence of where that livelihood occurs rather than of county itself, and an applicant from either county with non-seasonal income lands in a different segment. The test is counterfactual: does an otherwise identical borrower from a different county receive the same limit? If yes, the segment is tracking livelihood, not location, and the position is arguable — provided the reasoning, the tests and the residual disparity are documented before launch, and monitored afterwards.

Bibliography

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- Arthur, D. and Vassilvitskii, S. (2007). *k*-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is “nearest neighbor” meaningful? In *International Conference on Database Theory (ICDT)*, pages 217–235.
- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.
- Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L. (2001b). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.

- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.
- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35.
- Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.
- McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Melis, G., Dyer, C., and Blunsom, P. (2018). On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*.

- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* 28.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.
- van der Maaten, L. and Hinton, G. (2008). Visualizing data using *t*-SNE. *Journal of Machine Learning Research*, 9:2579–2605.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wattenberg, M., Viégas, F., and Johnson, I. (2016). How to use *t*-SNE effectively. *Distill*.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2):241–259.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.